

Master's Thesis

Improving GitHub Project Recommendations

A Hybrid Model Using Content-Based NCF and LETOR with XGBoost

CHIAYU, YANG

Student number: 15468666
Date of final version: August 13, 2025
Master's programme: Data Science and Business Analytics
Specialization: Data Science
Supervisor: Prof. dr. C. Amrit
Second reader: Prof. dr. G. Capelleveen



FACULTY OF ECONOMICS AND BUSINESS

Abstract

Open-source platforms such as GitHub offer tremendous opportunities for developers to learn, build experience, and contribute to impactful software projects. However, the vast number of available repositories makes it difficult for developers to find suitable projects to contribute to. This challenge can lead to disengagement, and studies show that only around 2% of GitHub users become long-term contributors. A more effective recommendation system could help retain talented developers by directing them toward meaningful and engaging projects.

This thesis presents a two-stage recommendation framework for GitHub project suggestions. To capture developer interest and prioritize relevant projects, we define the strength of user interest using seven types of interaction behaviors. The first stage focuses on candidate generation using a Neural Collaborative Filtering (NCF) model. To enrich project representation, we evaluated three project description embedding techniques: TF-IDF, BERTopic, and BERT, and applied clustering algorithms (K-Means, Hierarchical, DBSCAN, and HDBSCAN) to extract topic-level features from project descriptions. To further refine recommendations, we apply a Learning-to-Rank (LETOR) approach using an XGBoost model to re-rank the candidate list based on richer semantic, behavioral, and social features.

Experimental results show that using BERT-based embeddings combined with DBSCAN clustering provides the highest quality topic features. In the one-stage recommendation setup, the best configuration reaches a Recall@10 of 0.08622. When combined with a LETOR using XGBoost, the two-stage pipeline significantly improves performance, reaching a final Recall@10 of 0.21193. These results demonstrate that our recommendation system effectively integrates semantic, behavioral, and social signals to deliver personalized OSS project suggestions.

Contents

1	Introduction	7
2	Background	9
2.1	Overview of GitHub	9
2.1.1	Interaction Data Recorded on GitHub	9
2.2	Challenge on GitHub platform	10
2.3	Factors Influencing Developers' Interaction with Open-Source Projects	10
2.4	Basic Flow of Recommender Systems	11
2.5	Recommendation System Research Overview	11
3	Methodology and Techniques	13
3.1	Content-based Clustering	13
3.1.1	Project Representation Techniques	13
3.1.1.1	TF-IDF with PCA Reduction for Project Description Encoding	13
3.1.2	Clustering Algorithms	15
3.1.3	Clustering Performance Evaluation Metrics	17
3.2	Recommendation System	17
3.3	Candidate Generation	18
3.3.1	Neural Network-based Collaborative Filtering (NCF)	18
3.4	Ranking	20
3.4.1	Learning to Rank(LETOR) with XGBoost	20
3.5	Evaluation	21
3.6	Recommendation System Baseline Approach	22
4	Data Collection and Preparation	23
4.1	Exploring Data Sources	23
4.2	GHTorrent Dataset Overview	24
4.2.1	Data Schema	24
4.3	Interaction Type Data on GitHub	25
4.4	Data Collection and Process	26
4.5	Feature Extration	27
5	Experiments	30
5.1	Overview of Clustering method	30

5.2	Clustering Methodologies for Project Grouping	31
5.2.1	Unsupervised Clustering via TF-IDF and Dimensionality Reduction	31
5.2.2	Semantic Clustering via Pre-trained Language Models: BERT	32
5.2.3	Topic Modeling for Project Grouping via BERTopic	33
5.2.4	Clustering and Performance Evaluation	33
5.2.4.1	K-Means Clustering	33
5.2.4.2	Hierarchical Clustering	33
5.2.4.3	DBSCAN Clustering Analysis	34
5.2.4.4	HDBSCAN	35
5.3	Candidate Generation: Neural Collaborative Filtering	37
5.4	Ranking: Learning-to-Rank (LETOR) with XGBoost	41
5.5	Baseline Selection	41
6	Evaluation and Result	42
6.1	Constructing the ground truth	42
6.2	Evaluation of the Recommendation System	42
6.2.1	One-stage Recommendation System	42
6.2.2	Baseline Performance Comparison	43
6.2.3	Two-Stage Recommendation System	43
6.2.4	Training Time Discussion	44
6.2.5	XGboost Feature Importance	46
6.3	Research Question Result	47
7	Conclusion and Future Work	49
7.1	Conclusion	49
7.2	Future Work	50
	Appendices	51
A	GHTorrent Google BigQuery url	51
B	Cluster Distribution	52
C	BERT Distribution	54
	Bibliography	55

List of Figures

2.1	Basic flow of recommendation system	12
3.1	The workflow of the BERTopic model	15
3.2	Recommendation system architecture showing the two-stage workflow, where candidate projects are retrieved and subsequently ranked before presenting a personalized selection to the user.	18
3.3	Workflow diagram for LETOR training	20
4.1	GHTorrent Schema Entities	25
4.2	Summary of interaction data and categorical feature distribution in the decision training .	29
4.3	Token count distribution across each description after data processing.	29
5.1	Cumulative Variance Retention in PCA at Key Thresholds (90%, 80%, 70%)	32
5.2	RMSE Trend Across Principal Components	32
5.3	Silhouette Scores for k-means Clustering with TF-IDF tokenized project descriptions . .	34
5.4	Silhouette Scores for k-means Clustering with BERT-Based tokenized project descriptions	34
5.5	Silhouette Scores for Hierarchical Clustering with TF-IDF tokenized project descriptions	35
5.6	Silhouette Scores for Hierarchical Clustering with BERT-Based tokenized project descriptions	35
5.7	Silhouette Scores for DBSCAN Clustering with TF-IDF tokenized project descriptions .	36
5.8	Silhouette Scores and Davies-Bouldin Score for DBSCAN Clustering with BERT-Based tokenized project descriptions	37
5.9	HDBSCAN as the clustering method with TF-IDF tokenized project descriptions	37
5.10	HDBSCAN clustering results using UMAP projection of BERT-based tokenized project descriptions. A total of 46 clusters were identified. The left plot displays data points with white edges, while the right plot omits edges for a cleaner contrast.	38
5.11	Architecture of the NCF-based candidate generation model, showing embedded sparse features concatenated with continuous features. Deep candidate generation model architecture showing embedded sparse features concatenated with dense features.	40
6.1	XGBoost Feature Importance	47
B.1	GitHub Project Distribution Across 47 Clusters Generated via TF-IDF and K-Means Method	52
B.2	GitHub Project Distribution Across 8 Clusters Generated via BERT and DBSCAN Method	53

C.1 HDBSCAN clustering results using UMAP projection of BERT-based tokenized project descriptions.	54
--	----

List of Tables

4.1	Statistics summarizing developers' interactions with multiple projects. Each row presents the number of developers, the total onboarding decisions they made, and the corresponding percentage calculated from 350,000 studied onboarding cases.	29
5.1	Comparison of Clustering Algorithms Using Unsupervised Metrics with TF-IDF and PCA for Topic Modeling	38
5.2	Comparison of Clustering Algorithms Using Unsupervised Metrics on BERT-Based tokenized project descriptions	38
5.3	Comparison of HDBSCAN Clustering Performance under Different Parameters on BERT-Based Project Description	39
5.4	Comparison of BERTopic with HDBSCAN Using Different Transformer Embeddings and Clustering Parameters Evaluated by Unsupervised Metrics	39
6.1	Recall@K Accuracy Comparison for Generated Models(NCF model) and Benchmark Methods	45
6.2	NDCG@K Accuracy Comparison for Generated Models(NCF model) and Benchmark Methods	45
6.3	Accuracy Comparison of Proposed Two-Stage Models (NCF with XGBoost) and Benchmark Methods Across (a) Recall@K and (b) NDCG@K	45
B.1	Project Clustering Key Word Using TF-IDF and K-Means: 47 Group Overview	52
B.2	Project Clustering Key Word Using BERT and DBSCAN: 8 Group Overview	53

Chapter 1

Introduction

Open-source software (OSS) development has become a driving force behind global technological innovation. The decentralized collaboration feature of OSS empowers developers worldwide to contribute to projects, exchange knowledge, and drive the advancement of software ecosystems (Vasilescu et al., 2015). GitHub hosts millions of repositories and facilitates large-scale contributions across diverse domains (Bai et al., 2022). However, with the large number of resources, finding the right project remains a challenge for both developers and project maintainers.

For developers, identifying OSS projects that align with their skills, interests, and career goals can be overwhelming (Hevner et al., 2004). Faced with an abundance of repositories and limited guidance, many resort to manual browsing or rely on social cues, which often leads to mismatches or abandoned contributions. From the maintainers' perspective, attracting skilled contributors is equally difficult. Many high-quality but lesser-known projects suffer from low visibility, which hinders their growth and limits their impact. Moreover, despite initial enthusiasm, contributions often remain unsatisfactory, requiring developers to repeatedly revise and review their code to meet project standards and goals. This cycle can be frustrating for both sides (Steinmacher et al., 2015). Ultimately, project owners will stop accepting new commits, and contributors will leave the platform. The fact that only 2% of contributors become long-term participants (Eluri et al., 2021) further proves that GitHub needs a smarter, more personalized project recommendation mechanism.

GitHub's rich ecosystem offers more than just code hosting. It provides structured and multifaceted data on developer behavior, including stars, follows, issues, pull requests, and 12 more (Chacon and Straub, 2014). These interaction patterns, when combined with social network features, offer deep insights into user preferences and community dynamics. Prior research has demonstrated that leveraging social connections can help infer developer interests and improve recommendation quality (Hu et al., 2018). GHTorrent, the dataset archive GitHub metadata at scale, further enables the construction of structured interaction graphs essential for data-driven modeling. In our study, we focused on seven types of interaction behaviors to represent varying degrees of developer interest: star, follow, fork, issue, pull request, pull request comment, and commit. The lightweight signals to high-involved engagement, allowing the model to distinguish between different levels of commitment and intent.

The purpose of the thesis is to design and develop a developer-project recommendation system that bridges the gap between project visibility and contributor discovery. We model GitHub as a dynamic interaction graph, where developers and projects are connected through behavioral signals such as stars, forks, issues, and code contributions. Our system goes beyond traditional collaborative and content-based filtering by incorporating deep learning and learning-to-rank techniques.

The recommendation pipeline in this study adopts a two-stage framework (Covington et al., 2016). In

the first stage, we employ Neural Collaborative Filtering (NCF) to learn latent developer–project interactions from structured GitHub data. These predictions are then enriched with semantic insights extracted from project descriptions using advanced language models. To capture thematic structure, we further apply clustering algorithms—including K-means, Hierarchical Clustering, DBSCAN, and HDBSCAN—to group repositories based on their underlying topics. In the second stage, we adopt XGBoost with the LambdaMART objective as a learning-to-rank method. This model integrates both semantic representations and the interaction confidence scores produced by the NCF model to optimize recommendation quality. By leveraging feature relevance and ranking metrics, our pipeline effectively combines behavioral, semantic, and structural signals to deliver high-quality personalized recommendations.

Experimental results demonstrate that the NCF model combined with LETOR using XGBoost achieves the best performance, with Recall@10 and Recall@50 scores of 0.2119 and 0.3008, respectively, significantly outperforming all baseline models. Notably, compared to the second-best model, which uses TF-IDF tokenization with K-Means clustering, our BERT-based approach with DBSCAN achieves relative improvements of 20.48% at Recall@10 and 12.28% at Recall@50, and 27.15% and 44.38% at NDCG@10 and NDCG@50, respectively.

By synthesizing deep learning, natural language processing, and ranking algorithms, our system produces recommendations that are not only personalized and scalable but also more accurate in capturing developer intent. This solution directly tackles the dual challenge of boosting developer engagement while amplifying the visibility of high-potential open-source projects, fostering longer engagement, and reducing contributor churn.

The main contributions of this work are as follows:

1. We develop a BERT-based recommendation system combined with DBSCAN clustering to help GitHub contributors discover suitable projects for meaningful engagement, particularly those that lead to frequent contributions, such as commits.
2. We design a set of ten features to capture developer behavior and preferences. We further demonstrate that incorporating rich semantic representations from BERT and confidence scores from the NCF model significantly enhances the ranking performance.
3. We perform a detailed feature importance analysis and find that personal behavioral and connection signals are more predictive than static metadata, making them key drivers of successful developer–project matching.

The rest of the paper is organized as follows: Chapter 2 presents the background and motivation for the study. Chapter 3 introduces the methodologies and technical approaches. Chapter 4 outlines the dataset and details the data collection process. In Chapter 5, we describe our modeling framework, training methodology, and experimental design. Chapter 6 evaluates the results in light of the research questions. Finally, Chapter 7 concludes with the key findings and suggests directions for future work.

Chapter 2

Background

Before presenting the design of our recommendation system, this chapter provides a brief overview of the GitHub platform, focusing on its structural components interaction data that form the basis of our system. We also present the general architecture of recommendation pipelines commonly reviewed in several academic literatures (Ricci et al., 2011; Jawansingh and Rai, 2025) and adopted in online platforms like YouTube¹ (Covington et al., 2016) and Pinterest².

2.1 Overview of GitHub

GitHub³ is a widely used web-based platform for hosting Git repositories and serves as a central hub where millions of developers collaborate on countless projects. It hosts a significant portion of all existing Git repositories and supports essential tasks such as code hosting, issue tracking, version control, and peer code reviews, making it a primary tool in the open-source ecosystem (Chacon and Straub, 2014).

GitHub operates as a rich social network (Lima et al., 2014) that records diverse developer behaviors. Users can build personalized profiles that showcase their contributions, provide self-introductions, link to external websites, and optionally include details like location and company affiliation (Dabbish et al., 2012). Social interactions are embedded throughout the platform, allowing users to comment on code activities, react to issues, star repositories, and follow other developers to stay updated on their activities.

2.1.1 Interaction Data Recorded on GitHub

GitHub provides accessible data sources through its official API, which continuously records users' public interactions, including commits, forks, pull requests, issue activity, and repository events (GitHub Documentation Team, 2022). These interaction records have become a valuable resource for researchers and scientific institutions, facilitating the study of developer behavior, skillsets, and preferences (Onoue et al., 2013), and powering the development of data-driven recommendation systems (Zhang et al., 2014). Core functionalities such as pushing and pulling code, forking repositories to create derivative versions, and managing project workflows via an integrated issue tracking system enable efficient and transparent collaboration. These features allow developers to report questions, suggest enhancements, and stay aligned across distributed teams, making GitHub one of the most effective platforms for software development and coordination.

In our work, we leverage this interaction data to design a system that recommends potential collaborators on GitHub. We refer to these behavioral records as *interaction-type data*, which encapsulate not only technical contributions but also social engagement signals.

¹<https://www.youtube.com/>

²<https://www.pinterest.com/>

³<https://github.com/>

At the time of writing, GitHub is home to more than 420 million public repositories and serves over 150 million developers around the world ⁴, making it the largest open-source platform in the world. The platform’s detailed records of user information, repository metadata, and rich interaction logs offer a robust foundation for building recommendation systems tailored to individual developer interests and collaboration patterns.

2.2 Challenge on GitHub platform

OSS projects rely heavily on contributors (Nourry et al., 2024). While platforms such as GitHub have experienced rapid expansion and widespread adoption (Hu et al., 2018), sustained contributor engagement remains a pressing issue. Studies (Eluri et al., 2021) reveal that nearly 98% of developers discontinue participation before reaching a three-year tenure, with only a small fraction evolving into long-term contributors (LTCs). This high attrition rate poses a fundamental challenge to the continuity and resilience of OSS ecosystems.

There are several factors that influence whether contributors remain active over time, including individual expertise, documentation quality, leadership structure, and the composition of the development team (Blincoe et al., 2016). Beyond these, discoverability remains a major challenge. Navigating through thousands of repositories to find a compelling project is time-consuming, and deciding whether to commit requires careful consideration (Steinmacher et al., 2015). The dynamic and ever-changing nature of developer interests (Jiang et al., 2019), combined with the fact that many GitHub groups consist of developers from cross-disciplinary backgrounds (Li et al., 2017), makes it inefficient to rely solely on a single user group or interest cluster when seeking suitable projects. This highlights the need for smarter and more personalized recommendation strategies.

To improve this experience, it is essential to help developers connect with projects that match their skillsets and interests within a welcoming community. If contributors can discover meaningful projects within the first few repositories they browse, GitHub would become a more enjoyable and supportive environment—ultimately retaining more talent and fueling innovation across the platform.

2.3 Factors Influencing Developers’ Interaction with Open-Source Projects

When developers choose to engage with OSS projects, whether by contributing code, reporting issues, or following updates, their decisions are influenced by a combination of practical needs and perceived project qualities. Prior research has identified several key factors that shape these interaction behaviors. First, technical factors such as the programming language and the quality of the codebase play a significant role in determining a project’s accessibility and maintainability (Li et al., 2022). Developers are more likely to interact with projects whose technical attributes align with their expertise and preferences. Second, the background similarity between the developer and the contributor, especially the project owner, can increase the likelihood of interaction. Developers are more engaged when they share similar experiences or technical interests with team members. Social proximity also plays a role; those who know someone in the project are more likely to participate (Hahn et al., 2008). According to Bai et al. (2022), 90% of developers will Star a project when such a connection exists, and 60% will go on to take further actions like opening Issues, submitting Pull Requests, or making Commits. Additionally, social motivations such as staying informed about project activity, discovering new trends, learning

⁴<https://github.com/about>

from others, and maintaining professional connections drive developers to follow and interact with OSS projects (Kobayakawa and Yoshida, 2019). Furthermore, project responsiveness like how quickly and effectively maintainers respond to issues or pull requests, and characteristics like testability, sustainability, and trustworthiness are critical in shaping engagement. Projects that are easier to modify and demonstrate ongoing maintenance tend to attract more frequent and meaningful contributions.

2.4 Basic Flow of Recommender Systems

Recommender systems typically follow a standardized pipeline composed of several key stages (Ricci et al., 2011; Jawansingh and Rai, 2025). The complete process of building and maintaining a data-driven recommendation framework is illustrated in Figure 2.1.

The system begins by selecting a qualified dataset that contains a sufficient volume of user profiles, historical interactions, and contextual signals such as ratings or behavioral logs. This is followed by **data preprocessing**, which includes procedures such as cleaning, normalization, and feature extraction to structure the data for downstream tasks. Subsequently, the system proceeds to the **model selection and training** stage. At this stage, there are various algorithms such as Neural Collaborative Filtering (NCF) (He et al., 2017), Graph Neural Networks (GNN) (Gao et al., 2023), or hybrid architectures (Seth and Sharaff, 2022), may be employed to learn user and item representations from the processed data. The goal is to capture latent preferences and behavioral patterns. To evaluate the learned models, **performance metrics** such as Recall@K, Precision@K, and Normalized Discounted Cumulative Gain (NDCG) are used to quantify both accuracy and ranking quality. These evaluations guide model tuning and enable fair comparison with baseline methods.

Once the model is optimized, it is ready to generate personalized recommendations for users. Simultaneously, a **feedback loop** is built in to improve the system over time. This loop collects user reactions—such as clicks, ratings, or skips—through the user interface and feeds them back into the model during retraining. This cycle helps the system adjust to changing user preferences, making recommendations more relevant in fast-moving environments.

2.5 Recommendation System Research Overview

We reviewed substantial research on recommendation systems; however, only a few studies have specifically addressed the problem of GitHub project recommendation. Collaborative filtering (CF)-based and network-based approaches are two widely used approaches in research (Raza et al., 2025). Furthermore, recent large-scale recommendation systems often adopt a two-stage framework, consisting of candidate generation followed by ranking, to improve scalability and accuracy (Covington et al., 2016).

Early CF-based approaches largely relied on project-level similarity. For example, Koskela et al. (2018) proposed a hybrid recommendation framework by combining multiple similarity measures derived from different project features to form recommendation lists. Similarly, Raza et al. (2025) explored popularity-based methods, using metrics such as the number of stars to project and recommend high-quality projects. While these approaches are effective to some extent, they emphasize project-centric features and often overlook the personal interests of developers. To address this limitation, recent studies have shifted toward personalized recommendations, integrating behavioral signals from developer interactions. Yang et al. (2021) introduced a deep matrix factorization model that uses individual user behavior to better match developers with relevant projects. Likewise, Guendouz et al. (2015) modeled GitHub user-

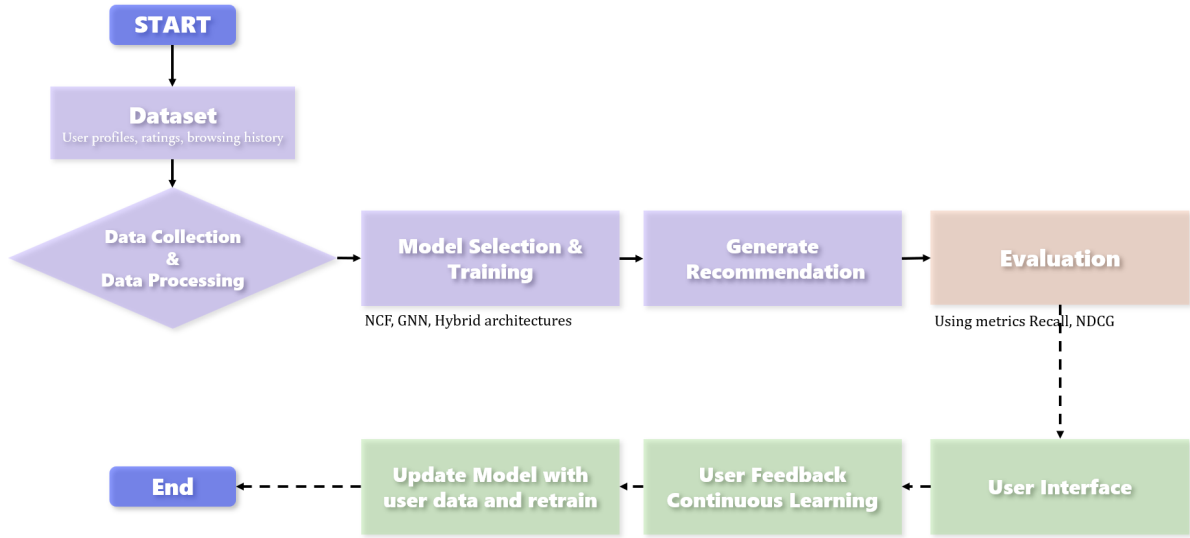


Figure 2.1: Basic flow of recommendation system

repository interactions as a user-item matrix and applied CF techniques to calculate similarity and generate recommendations. Further studies by Bai et al. (2022) and Han et al. (2019) investigated various interaction types (e.g., stars, forks, follows) to determine which behaviors are most predictive of future contributions.

From a network-based perspective, researchers have explored graph-based representations to model structural relationships between developers and projects. Matek and Zebec (2016) built a bipartite graph using GitHub activity data, and applied unsupervised models to predict future user-project connections. Di Rocco et al. (2023) proposed a hybrid system that combines collaborative filtering, stochastic modeling, and Conditional Bayesian Networks (CNBN) to capture both observable and latent relationships. Recent advancements in deep neural ranking models have further boosted recommendation accuracy. For example, Liu et al. (2018a) introduced NNLRank, a list-wise LETOR framework that optimizes repository rankings based on users' interaction history. Likewise, ILMART (Lucchese et al., 2025) builds upon LambdaMART to provide a fast and interpretable ranking model, improving overall system performance without compromising on efficiency.

Building on insights from these studies, we develop our two-stage recommendation system tailored to the GitHub platform ecosystem.

Chapter 3

Methodology and Techniques

We first introduce the Design Science Research framework that we follow throughout the whole research. Break it down into the Relevance Cycle, Design Cycle, and Rigor Cycle, emphasizing how these stages guide the research. This study adopts the Design Science Research (DSR) framework proposed by Hevner et al. (Hevner et al., 2004), serving as a methodological foundation for the design, development, and evaluation of a recommendation system tailored for OSS projects.

The Relevance Cycle focuses on identifying and understanding the real-world challenges encountered by developers and OSS maintainers in discovering and recommending suitable projects. These practical needs inform the problem definition and guide the system requirements. The Design Cycle encompasses the iterative process of developing and refining the recommendation models. In this phase, deep learning architectures, natural language processing (NLP) techniques, and ranking methods are integrated to optimize the system's performance and address the research objectives. The Rigor Cycle ensures the theoretical soundness and empirical validity of the proposed solutions. Models are benchmarked against established recommendation systems and evaluated using standard metrics to demonstrate their effectiveness and generalizability.

By following the DSR framework, this research aims to bridge the gap between theoretical advancements and practical applicability in the domain of open-source project recommendation.

3.1 Content-based Clustering

To enable effective understanding of textual data using machine learning techniques, several encoding models have been developed (Taha et al., 2024; Jamshidi et al., 2024). In this research, we focus on three representation methods utilized in our experiments to encode project descriptions: TF-IDF, BERT-based embeddings, and BERTopic. Each approach offers distinct mechanisms for capturing semantic features, and we describe their underlying principles and the role they play in facilitating project clustering tasks within our recommendation framework.

3.1.1 Project Representation Techniques

3.1.1.1 TF-IDF with PCA Reduction for Project Description Encoding

Term Frequency–Inverse Document Frequency ($TF - IDF$) is a foundational technique in information retrieval, designed to quantify the significance of individual terms within a document relative to a larger corpus. It assigns weights to words based on two components: Term Frequency (TF) and Inverse Document Frequency (IDF). One key advantage of TF-IDF lies in its ability to accommodate document length variation, mitigating biases that arise from disproportionately frequent terms in longer texts (Wu et al., 2008). This ensures that uniquely informative keywords are emphasized, while commonly occurring terms are appropriately downweighted.

- **Term Frequency (TF):** Measures how frequently a term t appears in a document d . To normalize for varying document lengths, the term frequency is calculated as:

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (3.1)$$

where $f_{t,d}$ denotes the frequency of term t in document d .

- **Inverse Document Frequency (IDF):** Evaluates the rarity of a term across the corpus. It reduces the weight of common terms while increasing that of rare ones:

$$IDF(t) = \log \left(\frac{N}{|\{d \in D : t \in d\}|} \right) \quad (3.2)$$

where N is the total number of documents and the denominator counts the number of documents in which term t appears.

- **TF-IDF:** The formula of TFIDF can be written as:

$$TFIDF(t, d) = TF(t, d) \times IDF(t) \quad (3.3)$$

emphasizes terms that are important to a specific project or domain while reducing the influence of frequently occurring words.

Principal Component Analysis (PCA)

We employ PCA as a main technique for feature extraction and dimensionality reduction (Hegde et al., 2023). PCA improves the computational efficiency and the interpretability of machine learning models, especially beneficial in clustering and classification tasks.

PCA achieves dimensionality reduction by applying an orthogonal linear transformation that converts correlated variables into a new set of uncorrelated variables, known as principal components. It reduces the dimensionality of the original $X_{m \times n}$ data matrix by preserving most of the variance in fewer components. To compute the principal components, we use Singular Value Decomposition (SVD) for its speed and numerical robustness. SVD decomposes X into $U\Sigma V^T$, where the top k components from U are selected to project the data into a lower-dimensional space.

After PCA, we apply a whitening transformation (Kale et al., 2018) to standardize the variances of the components. Whitening decorrelates the features and scales them to unit variance, which helps to suppress noise and prevent dominant components from skewing the analysis. This transformation results in a normalized feature space suitable for effective clustering.

Sentence-Level Embeddings with BERT

The BERT¹ is a transformer-based language model introduced by Devlin et al. (2019) and developed by Google AI. It is pre-trained on a massive corpus using two key unsupervised tasks: Masked Language Modeling and Next Sentence Prediction. BERT has proven state-of-the-art performance in semantic understanding tasks, effectively capturing nuanced linguistic and contextual relationships between words

¹Bidirectional Encoder Representations from Transformers

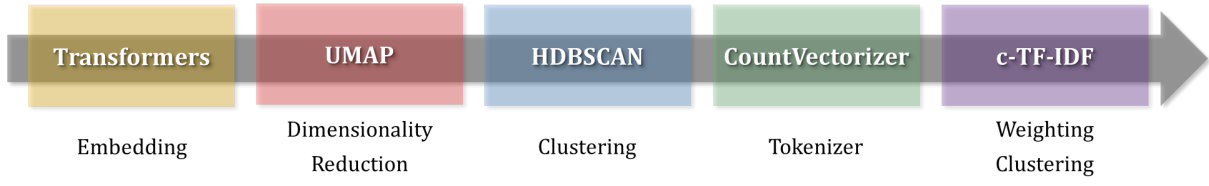


Figure 3.1: The workflow of the BERTopic model

and phrases Garrido-Merchan et al. (2023). This makes it a robust choice for generating semantic embeddings of OSS issue descriptions without requiring task-specific fine-tuning.

Topic Modeling via BERTopic

BERTopic (Grootendorst, 2022) is a topic modeling technique that integrates transformer-based embeddings with class-based TF-IDF scoring. This approach allows for semantically meaningful topic generation and improved interpretability compared to traditional topic modeling methods.

Specifically, BERTopic utilizes pre-trained sentence embeddings² to represent each project description in a high-dimensional semantic space. To enable efficient clustering, the embeddings are first reduced in dimensionality using UMAP, a non-linear technique that preserves local structure and is well-suited for textual data. Next, a clustering algorithm such as HDBSCAN is applied to group similar documents together without requiring a predefined number of clusters.

Once clusters are formed, BERTopic applies class-based TF-IDF to extract the most representative keywords for each topic. This enhances topic interpretability by ensuring that important terms are ranked based on their relative significance within each cluster, rather than across the entire corpus. The overall architecture of the BERTopic pipeline used in our approach is illustrated in Figure 3.1.

In this work, we adopt BERTopic with two configurations (Hegde et al., 2023):

- **Custom Pipeline:** We use the "all-MiniLM-L6-v2"³ model from SentenceTransformers, which is optimized for semantic similarity tasks and performs well on short texts such as GitHub project descriptions. This setup allows fine-tuned control over the dimensionality reduction and clustering steps, enabling more precise topic extraction tailored to the dataset.
- **Default Configuration:** The pre-configured default BERTopic pipeline use "distiluse-base-multilingual-cased-v2"⁴ model. This setup prioritizes ease of integration, while preserving semantic coherence during topic formation. We use this configuration as a baseline for comparison with our custom model.

Unlike traditional clustering, BERTopic emphasizes topic separability and interpretability, which is particularly useful when recommending projects with explainable context.

3.1.2 Clustering Algorithms

Clustering is an unsupervised machine learning technique that identifies patterns and groups similar data points together without requiring labeled examples (Ezugwu et al., 2022). In our setting, clustering

²Sentence-BERT or MPNet

³<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

⁴<https://huggingface.co/sentence-transformers/distiluse-base-multilingual-cased-v2>

serves to uncover latent topical or functional similarities between repositories based on their project descriptions and programming language. By grouping repositories into coherent clusters, we enable more context-aware and interest-aligned downstream recommendations.

We evaluated 4 standard clustering algorithms, including K-Means, Hierarchical Clustering, DBSCAN, and HDBSCAN, to determine which methods best capture the semantic similarity in our project.

- **K-Means Clustering:** K-Means is one of the most widely used clustering algorithms due to its simplicity, efficiency, and scalability to large datasets. It partitions the data into k non-overlapping clusters, where k is a predefined number of clusters. The algorithm begins by randomly selecting k initial centroids. Each data point will be assigned to the nearest centroid based on the Euclidean distance, and then the centroids will be updated to the mean of the points within each cluster. This process continues iteratively until the centroids converge. K-Means is known for its fast convergence to a local optimum, although it is sensitive to the initial choice of centroids. Prior research shows that k-means can be used for solving GitHub project clustering problems (Zahra et al., 2015; Mohamed, 2020).
- **Hierarchical Clustering:** Hierarchical clustering organizes data into a tree-like structure, making it useful for uncovering nested groupings and capturing clusters of varying shapes and sizes (Ezugwu et al., 2022). We applied a bottom-up agglomerative approach, where each data point starts as its own cluster and is iteratively merged with others based on a linkage criterion. This process continues until all points form a single cluster or a stopping condition is met. The merging is guided by a linkage metric, which defines the distance between clusters rather than individual data points.

In the study by Nalini et al. (2023), they compared K-Means and hierarchical clustering methods. The results showed that hierarchical clustering produced more coherent groupings, as evidenced by higher performance on SI and DBI within the context of a GitHub-based recommendation system.

- **DBSCAN⁵:** DBSCAN is a density-based clustering algorithm capable of identifying clusters of arbitrary shapes by distinguishing between dense and sparse regions in the data (Jin et al., 2023). DBSCAN does not require the number of clusters; instead, it relies on two parameters: the neighborhood radius ϵ , and the minimum number of points to form a dense cluster. A key strength of DBSCAN is the ability to detect outliers and treat them as noise, making it highly effective for datasets with irregular structures. Moreover, the algorithm is order-independent and produces consistent results without bias (Zhang et al., 2020; Chen et al., 2018), which makes it especially suitable for real-world applications.
- **HDBSCAN (Hierarchical DBSCAN):** HDBSCAN leverages the strengths of both hierarchical clustering and DBSCAN. It’s an extension of DBSCAN that constructs a hierarchy of clusters based on varying density levels and condenses it into a flat clustering. Unlike DBSCAN, HDBSCAN does not rely on a single global ϵ threshold. Instead, it evaluates all possible values with respect to a minimum cluster size to identify stable clusters across different density regions. This makes it especially effective for handling variable-density datasets—common in natural language tasks. HDBSCAN retains the ability to detect noise and produces more compact and semantically meaningful clusters than standard DBSCAN in our experiments, with improved stability and structure (Malzer and Baum, 2020).

⁵Density-Based Spatial Clustering of Applications with Noise

3.1.3 Clustering Performance Evaluation Metrics

To quantitatively evaluate clustering performance and determine the optimal number of clusters, we use three validation indices: SI, CHI and DBI. These metrics provide complementary insights into inter-cluster cohesion and inter-cluster separation (Enriquez-Loja et al., 2025; Nalini et al., 2023).

1. **Silhouette Index (SI):** Silhouette Index is a widely used internal validation criterion in unsupervised learning. For each data point i , the Silhouette value $s(i)$ evaluates its cohesion within its assigned cluster versus its separation from the nearest neighboring cluster. It is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (3.4)$$

where $a(i)$ is the average distance between i and other points in its cluster, and $b(i)$ is the average distance between i and points in the nearest different cluster.

The Silhouette value ranges from -1 to $+1$:

- A **higher score** (near to 1) indicates the data point is in the correct cluster.
 - A score near **0** means the data might belong to other cluster.
 - A **negative score** implies the data point may have been assigned to the wrong cluster.
2. **Calinski–Harabasz Index (CHI):** The CHI evaluates clustering quality by measuring the ratio between the dispersion of data points across clusters (between-cluster) and the dispersion within each cluster (within-cluster). It is calculated as:

$$CH = \frac{N - k}{k - 1} \times \frac{SS_B}{SS_W} \quad (3.5)$$

where N is the total number of data points and k is the number of clusters. SS_B and SS_W represent between-cluster and within-cluster variance, respectively. A higher CHI score indicates better-separated and more compact clusters.

3. **Davies–Bouldin Index (DBI):** The DBI measures the average similarity between each cluster and its most similar one, based on inter-cluster dispersion and inter-cluster separation (Davies and Bouldin, 1979). It is defined as:

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{S_i + S_j}{d(c_i, c_j)} \right) \quad (3.6)$$

Here, S_i is the average distance between the center of cluster i and all of its members. $d(c_i, c_j)$ is the distance between the centroids of clusters i and j . A lower DBI value indicates better clustering, as it reflects well-separated clusters. Ideally, the score should be close to 0.

3.2 Recommendation System

The overall architecture of our two-stage recommendation system is illustrated in Figure 3.2. The system begins with a candidate generation module, followed by a ranking module. The design is inspired by Covington et al. (2016) and Yue et al. (2018).

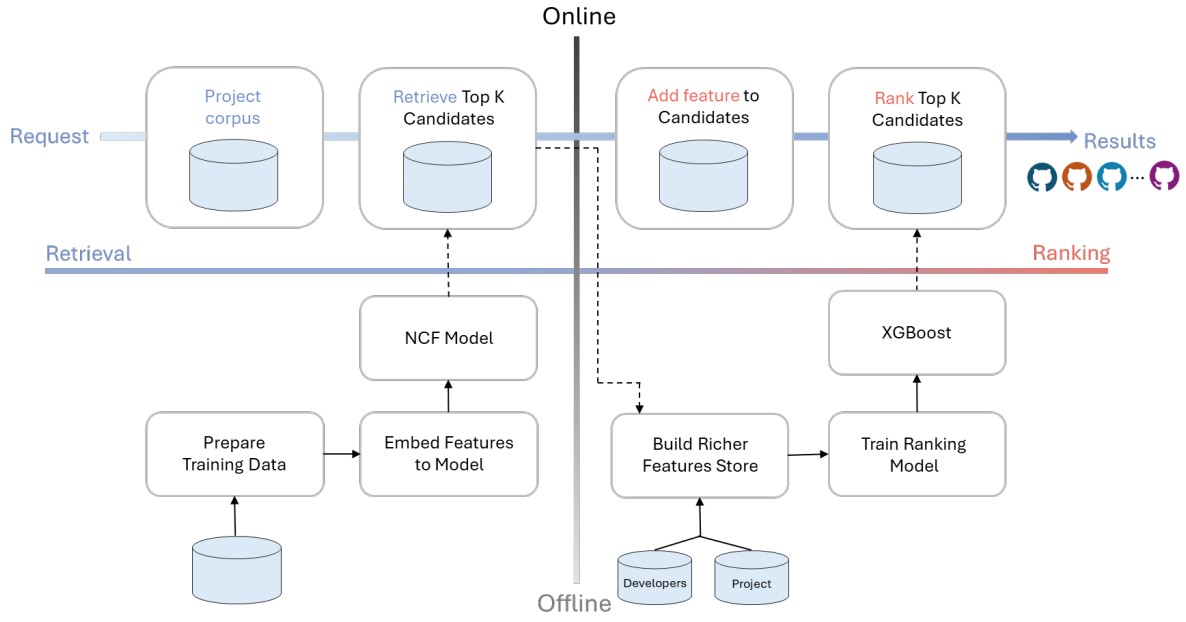


Figure 3.2: Recommendation system architecture showing the two-stage workflow, where candidate projects are retrieved and subsequently ranked before presenting a personalized selection to the user.

The candidate generation stage uses a **neural collaborative filtering (NCF)** model to learn latent representations of developer–project interactions. By capturing implicit feedback patterns along with developer features, NCF retrieves a subset of candidate projects (typically a few hundred) that are broadly relevant to the target developer. This stage emphasizes precision and leverages collaborative signals such as co-contribution histories and project engagement behaviors. In the second stage, we apply an **XGBoost-based Learning-To-Rank model** to re-rank the candidate projects. This model incorporates a rich set of engineered features. Unlike neural rankers, XGBoost offers faster training convergence and interpretable feature importance, making it suitable for re-ranking in production-grade setups.

This two-stage approach enables scalable retrieval from a large corpus of open-source repositories while ensuring that the final list is both personalized and contextually coherent. Moreover, this design allows for flexible integration of content-based signals and clustering-enhanced representations, which contribute to improved recommendation diversity and relevance.

3.3 Candidate Generation

During candidate generation, the enormous GitHub projects corpus is narrowed down to a few hundred potentially relevant projects for each developer. Candidate generation serves two main purposes: (1) It improves recommendation relevance by focusing on projects that are more likely to be of interest to each developer based on prior interactions and feature similarities, and (2) It reduces computational complexity during both training and inference (Covington et al., 2016).

3.3.1 Neural Network-based Collaborative Filtering (NCF)

To establish a foundational framework for developer–project recommendations on GitHub, we employ the NCF model. Collaborative Filtering (CF) is a well-established recommendation technique that suggests new items to users based on their past interactions and those of similar users (Fareed et al., 2023). In

our research, developers represent users, repositories (referred to as projects) are items, and interactions provide implicit feedback signals.

Model Architecture

The model begins by loading preprocessed data, including developer, project, and other features, which are then merged to form the model's input. The NCF architecture employs embedding layers to map developers and projects into dense vector representations, capturing their latent characteristics. These embeddings are concatenated and passed through multiple fully connected (dense) layers with ReLU activation functions, allowing the model to learn complex interaction patterns. Finally, a softmax-activated output layer to produce a probability distribution over multiple interaction classes. This setup allows the model to classify the type of interaction most likely to occur between a developer and a project, extending the NCF approach to multi-class settings.

Activation Function: Rectified Linear Unit (ReLU)

ReLU is employed as the non-linear activation function in all hidden layers of the recommendation model. It is defined as:

$$f(x) = \max(0, x) \quad (3.7)$$

Optimizer: Adam (Adaptive Moment Estimation)

We use the Adam optimizer, which adapts the learning rate for each parameter using estimates of first and second moments of the gradients. The parameter update rule is:

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (3.8)$$

By combining ReLU activation with the Adam optimizer, we achieve a balance between computational efficiency and expressive model capacity, making this configuration effective for large-scale recommendation tasks on GitHub (Kingma and Ba, 2015).

Output Layer and Loss Function

We employ a multi-class classification framework to model developer-project interactions, where the model outputs a score vector $\hat{y}_{di} \in \mathbb{R}^C$ for each developer-project pair (d, i) , with C representing the number of possible interaction classes. To convert these raw scores (logits) into a valid probability distribution over classes, we apply the softmax function:

$$P(c \mid d, i) = \frac{\exp(\hat{y}_{di}^{(c)})}{\sum_{k=1}^C \exp(\hat{y}_{di}^{(k)})} \quad (3.9)$$

To train the model, we use the standard multi-class cross-entropy loss is defined as:

$$\mathcal{L}_{di} = -\log \left(\frac{\exp(\hat{y}_{di}^{(c^*)})}{\sum_{k=1}^C \exp(\hat{y}_{di}^{(k)})} \right) \quad (3.10)$$

In the final step, the model generates a personalized list of recommended projects by matching the developer's profile with project features, which is then passed through a ranking model.

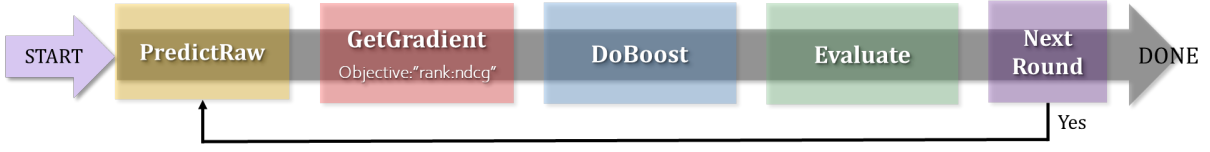


Figure 3.3: Workflow diagram for LETOR training

3.4 Ranking

Ranking plays a crucial role in surfacing projects that align with developer interests, elevating them to higher positions in the recommendation list. Using detailed data, this stage refines the candidate predictions and adapts the results to the specific user interface and presentation context (Covington et al., 2016). Unlike candidate generation, which operates on a large corpus, ranking involves scoring only a few hundred candidates per query. This computational efficiency allows the use of additional features. Compared to neural ranking models, XGBoost Learning to Rank offers practical advantages in terms of training efficiency (time and memory) (Bentéjac et al., 2021), resistance to overfitting through regularization and pruning, and interpretability via feature importance analysis. These strengths are especially prominent on tabular datasets, as discussed in recent benchmarks (Grinsztajn et al., 2022). Motivated by these findings, the second stage of our recommendation system adopts **Learning-to-Rank (LETOR)** algorithm using **XGBoost** as the ranking engine in our recommendation pipeline.

3.4.1 Learning to Rank(LETOR) with XGBoost

XGBoost is a widely adopted implementation of gradient boosting decision trees, originally introduced by Chen and Guestrin (2016). Beyond classification and regression, XGBoost supports LETOR tasks through objectives, which enable it to optimize ranking order. With appropriate configuration, these objectives can emulate LambdaMART-style learning by combining gradient boosting with ranking-specific gradient updates. The workflow of LETOR is illustrated in Figure 3.3

Overview of LambdaMART

LambdaMART has been demonstrated to be a highly effective algorithm for solving real-world ranking problems (Burgess, 2010). It combines the LambdaRank gradient formulation with Multiple Additive Regression Trees (MART) (Bengio et al., 2007).

- **LambdaRank** (Wen et al., 2025) extends RankNet by modifying the gradient signals to directly optimize ranking metrics with NDCG. It models the ranking problem through pairwise comparisons, where gradients are scaled according to the change in the ranking metric.
- **MART** (Friedman, 2001) builds an ensemble of regression trees in a stage-wise fashion, where each tree fits the residuals of the previous ensemble.

LambdaMART converts ranking metrics - NDCG into pairwise gradients by estimating the probability that one item should be ranked above another. Given a pair of items i and j , the pairwise probability is modeled with a sigmoid function:

$$P_{ij} = P(p_i > p_j) = \frac{1}{1 + \exp(-\sigma(s_i - s_j))} \quad (3.11)$$

where s_i and s_j are the predicted scores for items i and j , and σ is a scaling parameter that controls the shape of the sigmoid and is often set to 1. The learning objective encourages P_{ij} to align with the ground truth preference, penalizing misordered pairs.

Other pairwise ranking methods include RankSVM, which uses a hinge loss $\max(0, 1 - t)$ (where $t = s_i - s_j$), and RankNet, which minimizes the logistic loss $\log(1 + \exp(-t))$ s (Qomariyah et al., 2020). LambdaMART improves these approaches by directly incorporating ranking metrics: NDCG into the gradient computation while leveraging the high predictive power of boosted trees.

In our experiments, we adopt the `rank:ndcg` objective in XGBoost to train a LambdaMART-style model that assigns higher scores to relevant items, thereby generating personalized ranked lists for each developer query.

3.5 Evaluation

To evaluate the effectiveness of our recommendation model, we assess whether developers actually join the projects that were recommended to them. A developer is considered to have engaged with a recommended project if there is any subsequent interaction, such as starring, forking, or committing. Based on this criterion, we compute several standard top- N recommendation metrics to evaluate performance. We adopt the following evaluation metrics: To evaluate the effectiveness of our recommendation system, we adopt two widely used metrics: Recall@K and NDCG@K. While Recall@K captures the proportion of relevant items successfully retrieved among the top-K recommendations, NDCG@K reflects the ranking quality by assigning higher weights to correctly positioned relevant items. These metrics together provide a comprehensive assessment of recommendation accuracy and ordering performance.

1. **Recall@K:** Recall@K measures the proportion of relevant items that appear in the top- n recommendations. A higher recall indicates better coverage of relevant projects. In our experiments, we report Recall@10, Recall@50, Recall@100, and Recall@300 (Jawansingh and Rai, 2025).
2. **NDCG@K (Normalized Discounted Cumulative Gain):** NDCG is a widely-used metric for evaluating the quality of ranking algorithms. It extends the Discounted Cumulative Gain (DCG), which measures the ranking performance by penalizing incorrect ordering of relevant items. DCG considers both the relevance of an item and its position in the ranked list.

Intuitively, a model performs well if highly relevant items appear earlier in the list compared to less relevant ones. The DCG score at position p is computed as:

$$\text{DCG}_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)} \quad (3.12)$$

where rel_i denotes the relevance score of the item ranked at position i .

Since raw DCG values can vary across queries, it is normalized using the ideal DCG (iDCG)—the DCG of the perfect ranking order. NDCG ensures comparability across multiple ranking tasks and is defined as:

$$\text{NDCG}_p = \frac{\text{DCG}_p}{\text{iDCG}_p} \quad (3.13)$$

NDCG scores range from 0 to 1, with higher values indicating better ranking quality. In this method, we report NDCG@10, NDCG@50, NDCG@100, and NDCG@300.

These metrics together provide a comprehensive view of our model’s performance in terms of relevance and ranking quality.

3.6 Recommendation System Baseline Approach

Every recommendation system, no matter how complex and sophisticated its algorithms are, must be evaluated against well-established baselines to ensure its performance is both reliable and interpretable (Jawansingh and Rai, 2025). Baselines serve as essential reference points, helping distinguish genuine model improvements and enabling meaningful comparisons.

- **Random Baseline:** This is a non-personalized baseline, generating random recommendations for users. This serves as a lower bound for performance and verifies that the evaluation metrics are not biased by the structure of the dataset.
- **Top- N Most popular Baseline:** This approach is a non-personalized baseline that recommends the most popular items to every user. The popularity is determined by the total number of interactions observed in the training dataset (Anelli et al., 2022). This method captures the bias toward globally popular items and often performs surprisingly well in sparse datasets.
- **Neighborhood-based Recommendations:** This baseline can be split into two methods:
 1. **Item-based k NN:** Following the work by Sarwar et al. (2001), this method assumes that users who liked a particular item are likely to prefer similar ones. The method recommends items by identifying similarities in item attributes and user interaction patterns.
 2. **User-based k NN:** As outlined by Resnick et al. (1994), this method assumes users with similar interests will tend to prefer similar items. The algorithm identifies the k most similar users to a target user by analyzing behavior patterns. Recommendations are generated by aggregating the preferences of these neighboring users to suggest items the target user is likely to appreciate.
- **XGBoost-Based Baseline:** Inspired by the gradient-boosted decision tree framework introduced by Chen and Guestrin (2016), this supervised baseline models developer–project interactions as a pointwise prediction task. The feature set includes historical interaction statistics, repository meta-data, and user profile attributes. XGBoost’s ability to capture non-linear feature interactions and handle heterogeneous data types makes it a strong tabular benchmark for recommendation tasks.

Chapter 4

Data Collection and Preparation

In this chapter, we present the methodology used to construct our dataset for analyzing GitHub developer activity and project interactions. We begin by providing an overview of the various data sources considered, followed by a detailed explanation of our selection process and the rationale for choosing GHTorrent as the primary dataset. We then describe how we extracted, cleaned, and organized the data to support our downstream recommendation tasks.

To ensure comprehensive and scalable coverage of GitHub activity, we evaluated three publicly available datasets commonly used in open-source software mining: GitHub Archive, GHTorrent, and SOTorrent. Each of these sources provides unique perspectives and data schemas that influence their suitability for different types of analysis.

4.1 Exploring Data Sources

GitHub Archive, developed by Ilya Grigorik¹, is a streaming dataset that captures GitHub public event data such as issues, forks, commits, and pull requests in real-time, collected via the GitHub REST API and stored in hourly archives on Google BigQuery (Li et al., 2022). GH Archive offers high temporal granularity and is well-suited for time-series analysis of GitHub activity. However, its focus on events means it does not provide relational integrity across entities (e.g., no global user or repository tables), making it less ideal for building interaction graphs or tracking developer histories.

SOTorrent is a data set designed primarily to study the evolution of code snippets on Stack Overflow and their connections to GitHub repositories (Nugroho et al., 2022). It is particularly valuable for cross-platform research, such as analyzing code reuse between Stack Overflow and GitHub. The links in SOTorrent allow direct access to the corresponding Stack Overflow posts, offering insight into the types of questions developers ask and highlighting topic-specific interests. These activities—asking questions, reading answers, or participating in discussions—can serve as indicators of a developer’s technical focus and active involvement in particular technologies.

However, the dataset’s available records begin only from December 8, 2020, which creates a temporal gap compared to the GHTorrent. This misalignment limits the potential for directly integrating the two datasets in this study. While SOTorrent provides valuable contextual signals about developer interests, its limited timeframe makes it less suitable for modeling long-term engagement and rich data for a recommendation system.

GHTorrent, developed by Georgios Gousios and Diomidis Spinellis (Gousios and Spinellis, 2012), offers a comprehensive and relationally structured mirror of GitHub’s data, including users, repositories, issues, pull requests, comments, commits, and social links (stars, forks, followers, etc.). It regularly

¹<https://www.gharchive.org>

crawls and stores GitHub’s public data into a MySQL schema, enabling complex queries across interactions and temporal tracking of developer behavior. Compared to GH Archive and SOTorrent, GHTorrent provides the most suitable structure for constructing developer–repository interaction graphs, identifying long-term activity patterns, and integrating multi-type interactions for recommendation purposes.

4.2 GHTorrent Dataset Overview

One of the primary challenges in collecting large-scale GitHub data is the API rate limit imposed by GitHub, which restricts authenticated requests to 5,000 per hour (Gousios and Spinellis, 2012). To address this limitation, the GHTorrent project was developed as a scalable and continuously maintained data mirror of GitHub’s public activity. GHTorrent systematically crawled GitHub’s REST API and stored the retrieved data in two formats: an unstructured JSON-based MongoDB instance, and a relational MySQL database containing a structured and aggregated subset of the data.

The relational schema of GHTorrent differs significantly from the timeline-based GitHub Events API. Rather than storing entries chronologically, GHTorrent emphasizes entity relationships, providing normalized tables for users, repositories, issues, pull requests, commits, and other key interactions. This structure enables the construction of developer–repository graphs and the modeling of long-term behavioral patterns, which are essential for recommendation systems. However, we observed that the dataset does exhibit some limitations in terms of missing or inconsistent entries, such as unresolved commit authors, orphaned issue references, or incomplete pull request metadata. These data quality issues are discussed in Data Collection and Process

Another limitation is that the GHTorrent project is no longer actively maintained. The last publicly available snapshot of the MySQL dataset is from May 31, 2019, and is hosted on Google BigQuery under the table `ghtorrentmysql1906`². This dataset covers GitHub activity starting from October 29, 2007, but due to the incomplete nature of early GitHub data and our focus on temporal consistency and interaction volume, we restrict our analysis to the period from 2010 to 2018.

- **Training set:** January 1, 2010 – December 31, 2015
- **Validation set:** January 1, 2016 - December 31, 2018

This 2:1 ratio ensures both sufficient training data and a temporally coherent validation set, simulating real-world scenarios where models trained on historical developer behavior are applied to predict future preferences. This approach also reduces temporal leakage and supports reproducible evaluation of our recommendation pipeline.

4.2.1 Data Schema

The GHTorrent data schema, illustrated in Figure 4.1, mirrors the structure of the GitHub platform as exposed by its public REST API (Gousios and Spinellis, 2012; Tiburcio and Spohn, 2010). It captures key entities such as users, repositories, and their relationships, with most entities linked to a specific project and reflecting user-initiated actions. For example, pull requests are tied to repositories and created by users, while commits can span multiple repositories and forks. Dynamic objects like pull requests and issues are further tracked through state history tables that record their transitions over time. Most

²See Appendix A for full BigQuery URL



Figure 4.1: GHTorrent Schema Entities

records include a `created_at` timestamp to support temporal analysis of developer behavior. This schema provides a foundational understanding of developer–project interactions, enabling effective data extraction and preprocessing for recommendation modeling.

4.3 Interaction Type Data on GitHub

One of the main challenges in modeling developer preferences on GitHub is the lack of an explicit rating or scoring system. Unlike platforms such as Netflix³, GitHub users cannot give numerical scores to repositories. To address this, we rely on interaction records as behavioral signals, which reveal both the types of projects developers engage with and the depth of their engagement. These signals are essential for uncovering subtle developer preferences, particularly for building a recommendation system.

To better capture these variations, we propose a multi-level classification of interaction types, where each level reflects a different degree of interest and engagement. Following prior research (Bai et al., 2022), these interactions can be interpreted through the lens of three underlying motivational dimensions: exploration⁴, communication⁵, and cooperation⁶.

³<https://www.netflix.com/browse>

⁴Seeking repositories that align with current technical curiosity, even if they are niche or less popular.

⁵Engaging with others for feedback and dialogue, such as receiving comments on contributions or discussing issues.

⁶Actively collaborating with like-minded developers, often through co-development and shared ownership of codebases.

These dimensions are integrated into an ordinal framework, wherein higher levels correspond to stronger forms of engagement. We define this hierarchy based on typical GitHub interactions—ranging from low-effort signals such as starring a repository, to high-commitment activities like direct contributions through commits. Each ascending level signifies increasing effort, investment, and depth of involvement, thereby enabling more granular modeling of developer behavior. We define seven distinct interaction types, ordered from low to high engagement:

1. **Star ⁷ (Level 0, Exploration)**: A lightweight signal of interest or appreciation for a repository. It typically represents casual engagement without deeper exploration.
2. **Follow (Level 1, Exploration)**: Indicates sustained interest in a developer’s activity. By following someone, a user expresses intent to stay updated with their work, hinting at perceived credibility or alignment.
3. **Fork (Level 2, Communication)**: Represents a medium-level commitment. Developers fork repositories to experiment, adapt code, or prepare for contribution. Forking is often the first step before issuing pull requests and enables users to explore the project without needing collaborator access.
4. **Issue (Level 3, Cooperation)**: Captures user participation through bug reports, feature suggestions, or discussions. It demonstrates active participation by raising concerns or feature requests, reflecting involvement in project development.
5. **Pull Request (Level 4, Cooperation)**: A strong signal of technical contribution. Developers propose changes to the original codebase, triggering collaborative workflows involving code review, discussion, and negotiation with the maintainers.
6. **Pull Request Comment (Level 5, Cooperation)**: Involves providing feedback on code contributions. It indicates interpersonal collaboration, discussion, and mutual code understanding, often key in large-scale open-source communities.
7. **Commit (Level 6, Cooperation)**: Represents the highest level of engagement and ownership. A commit reflects direct modification of the codebase. As shown in prior research (Sun et al., 2017), commits provide strong signals of a developer’s domain expertise and working style, making them valuable for recommendation.

This interaction hierarchy enables us to model developer behavior in a more precise way. By incorporating the type and intensity of interactions, we improve the system to recommend projects that match the historical engagement styles and motivational patterns of users.

4.4 Data Collection and Process

To support this research, we utilized the GHTorrent dataset through Google BigQuery, which allowed efficient querying and processing of large-scale GitHub data. We analyzed six years of developer-project interactions between 2010 and 2015, focusing on projects written in eight widely used programming

⁷GitHub records this as a “Watch” event, which we relabel as “Star” for clarity.

languages: JavaScript, Java, Python, PHP, Ruby, C++, C, and C#, which were identified as popular languages according to GitHub statistics ⁸.

To ensure data quality, we applied several filtering and preprocessing steps. First, we removed non-English and insufficient project descriptions, which means descriptions with fewer than five words or containing any non-English characters will be deleted. These projects were considered likely to be personal, test, or inactive repositories (Liu et al., 2018a), which are less relevant for modeling realistic developer behavior. This threshold corresponds to the median length of collected project descriptions. Additionally, we excluded projects that did not have at least 2 watchers and 5 unique contributors, as such projects typically lack sufficient community engagement and are unlikely to attract newcomers. This criterion filters out abandoned or highly inactive repositories.

On the user side, we refined the developer dataset by eliminating fake or unreliable accounts using the `user_fake` flag, which marked accounts created for testing, bots, or those with data retrieval errors. We also filtered based on the `user_type` field, retaining only records labeled as standard user accounts (`user_type = 0`) and discarding organizational accounts (`user_type = 1`). This distinction ensures we focus on individual developer behavior rather than organizational contributions.

Figure 4.2 summarizes the training dataset comprising 350,000 successful interaction decisions involving 62,547 developers and 49,125 GitHub projects. With an extremely sparse rate of 99.98%⁹, each developer only interacts with a small fraction of the available projects. This level of sparsity is typical in recommendation system datasets and poses a key challenge for improving model accuracy. Our data covers developers across 8 programming languages, 44,929 companies, 8,579 geographical locations, and 162 countries, reflecting a wide diversity of developer backgrounds. This diversity highlights the potential of our recommendation system to support OSS contributors across various technical and geographical backgrounds.

4.5 Feature Extration

We extract 7 key features to represent developers' attributes and prior experience. These features collectively enhance the model's ability to understand developer behavior and provide accurate recommendations. All features are derived from the GHTorrent and enriched with developer-project-level metadata. In this section, we describe the definition and how we collect each of them in detail.

1. **Location:** The location is a developer-level attribute collected from GitHub user profiles via GHTorrent. It reflects regional preferences or constraints that may influence project participation. Prior studies show that user-location distribution can serve as a useful feature in collaborative filtering (Jiang et al., 2022).
2. **Country:** Similar to location, but analyzed on a broader geographical scale, providing a higher-level aggregation of regional traits and OSS development. Country data can further help identify global trends in developer engagement, reflect cultural or national-level preferences.
3. **Company:** We assume that developers working at the same company may share similar coding practices and collaboration culture, influencing their likelihood to join the same projects.

⁸Data can be found on GitHub: <https://github.info/>, accessed May 2025.

⁹Computed as $\text{sparsity} = 1 - \frac{\text{number of non-zero elements}}{\text{total number of elements}}$

4. **Programming Language:** This feature refers to the primary programming language used in a repository. It serves as a proxy for the technical domain of the project.
5. **Social Tie:** Developers are more likely to contribute to projects where they have previously collaborated with existing members (Nourry et al., 2024), highlighting the important role of social relationships in OSS communities. Prior research characterizes these communities as "pyramid meritocracies," where project owners often act as leaders and influence participation across the group (Blincoe et al., 2016). Popular or highly active users can guide their followers toward new repositories and shape broader collaboration patterns. To capture this effect, we define a *Social Tie* feature that quantifies the strength of prior interactions between a developer and the project owner. Specifically, we consider the number of members m_{it} who co-participated with the developer in previous projects. A smaller group suggests more direct and personal collaboration, indicating a stronger social connection. The feature is formally defined as:

$$Social = \frac{1}{m_{it}}$$

6. **Profile Similarity:** Developers are more likely to join projects where members share similar professional backgrounds, particularly company similarity. Such shared contexts can help new contributors adapt more quickly to a project's workflow and coding norms. We measure this by counting the number of project members who worked at the same company at the time the developer joined.

$$Profile_{sim} = P_i$$

7. **Technical Ability** Developers prefer contributing to projects written in languages they are proficient in (Liu et al., 2018b). To capture this, we define the technical ability score as the proportion of the developer's past contributions that match the language of the candidate project. For example, if a developer has contributed to 3 Python and 4 C++ projects, and the target project is written in Python, the technical ability score is:

$$Tech_{lang} = 3$$

Dataset	GitHub	Categorical Feature	Number
# of interaction data	350,000	Project Language	8
# of developers	62,547	Company	44,929
# of projects	49,125	Location	8,579
# of interaction type	7 (each takes 50,000 samples)	Country	162
Sparsity	99.9886%		

Figure 4.2: Summary of interaction data and categorical feature distribution in the decision training

Number of Interaction	Number of developer	Percentage
1	24963	39.91%
2	12502	19.99%
3	7458	11.92%
4	4793	7.66%
5	3231	5.17%
6	2211	3.53%
7	1571	2.51%
8	1160	1.85%
9	954	1.53%
>=10	3974	6.35%
Max	293	0.47%

Table 4.1: Statistics summarizing developers' interactions with multiple projects. Each row presents the number of developers, the total onboarding decisions they made, and the corresponding percentage calculated from 350,000 studied onboarding cases.

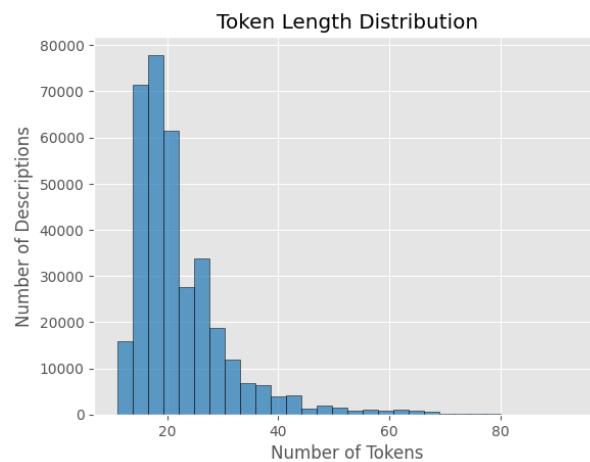


Figure 4.3: Token count distribution across each description after data processing.

Chapter 5

Experiments

This chapter presents the design and implementation of the machine learning models used in our recommendation system. We describe a two-stage pipeline that first employs a Neural Collaborative Filtering (NCF) model to generate candidate projects for each developer, followed by an XGBoost-based learning-to-rank model that reorders these candidates based on personalized relevance signals.

We begin by outlining the construction of training data and feature representations, followed by detailed descriptions of the architectures, training objectives, and evaluation metrics for each model.

5.1 Overview of Clustering method

The description is a concise summary of the repository; it is the key source of information, offering essential context to both human readers and machine learning models. It usually contains topic-indicating words. GitHub users who focus on repositories with similar topics tend to share mutual interests (Zhang et al., 2019).

To enhance the effectiveness and scalability of the recommendation system, we first group similar OSS projects into semantically coherent clusters. The underlying hypothesis is that projects sharing common topics, functionality, and application domain—are more likely to be relevant to similar groups of developers. By pre-clustering projects, the recommendation process can operate on a more granular, context-aware level, thereby reducing computational cost and increasing personalization accuracy.

In this study, each project is represented by a tuple in the form of (project id, description, programming language), where `project_id` serves as a unique identifier, `description` provides the project's topics and features, and `programming_language` reflects the project's technical domain. These heterogeneous features—textual and categorical—are transformed into unified vector representations to enable effective unsupervised clustering. Specifically, we examine three main pipelines. The first utilizes traditional NLP methods by representing project descriptions using TF-IDF, followed by dimensionality reduction via PCA, and subsequently applying clustering algorithms including K-Means, DBSCAN, Hierarchical Clustering, and Hierarchical-DBSCAN. The second pipeline replaces TF-IDF with BERT, a pre-trained deep contextual language model, to encode project descriptions into dense semantic vectors. Lastly, we explore BERTopic, a topic modeling approach that leverages transformer-based embeddings and class-based TF-IDF representations to produce semantically interpretable clusters.

By evaluating the quality and cohesion of the resulting clusters from each pipeline, we will determine which representation and clustering combination provides the most meaningful project groupings for downstream recommendation tasks.

5.2 Clustering Methodologies for Project Grouping

Following the approach by Bai et al. (2022), we initially applied TF-IDF to vectorize project descriptions. This method was chosen due to its demonstrated better performance compared to Word2Vec in similar GitHub-based recommendation tasks

5.2.1 Unsupervised Clustering via TF-IDF and Dimensionality Reduction

The first pipeline leverages traditional NLP techniques, TF-IDF, to capture the importance of terms relative to the corpus and group projects based on their descriptions. However, the TF-IDF matrix is inherently high-dimensional and sparse, posing challenges in terms of memory usage and noise. To address this, we apply PCA, a widely used linear dimensionality reduction method, to project the data into a lower-dimensional space while retaining the most informative semantic components. The resulting reduced feature vectors are then used as inputs to downstream clustering algorithms.

Text Preprocessing

Before applying TF-IDF for vectorization, we first cleaned and standardized the project descriptions through a text preprocessing pipeline. The pipeline included several key stages: data cleaning, case folding, noise removal, stemming, stop word removal, and length filtering (He et al., 2017). Initially, projects with missing, null, or trivial descriptions containing placeholders such as “test,” “demo,” or “dummy” were discarded to remove semantically irrelevant content. All remaining textual data were then standardized through case folding, converting descriptions to lowercase to avoid discrepancies in token recognition. Next, we performed noise removal, eliminating URLs, punctuation, code snippets, non-alphabetic characters, and excessive whitespace, which could hinder accurate tokenization. Following this, the Porter stemming algorithm (Porter, 1980) was applied to reduce words to their root forms, minimizing vocabulary size and consolidating semantically similar variants (e.g., “libraries” to “librari,” “running” to “run”). Finally, projects whose descriptions contained fewer than five words were excluded to mitigate sparsity in the TF-IDF matrix. These preprocessing steps ensured that the input data was consistent, noise-free, and semantically rich, thereby improving the reliability and interpretability of the clustering results.

TF-IDF Representation and Token Filtering

Following preprocessing, we applied the TF-IDF encoding scheme to transform the textual data into numerical feature vectors. Rather than arbitrarily fixing the vocabulary size, we used document frequency thresholds to constrain the token space. Initially, the corpus contained 29,074 unique unigrams. When including 2-gram phrases, to capture commonly co-occurring phrases like “open issue” or “data processing”, the number expanded to 182,109 tokens. Such a large vocabulary would produce a high-dimensional matrix that consumes over 66.7 GB of memory if left unfiltered.

To manage sparsity and improve computational feasibility, we applied two document frequency filters. First, we removed infrequent terms appearing in fewer than 20 project descriptions, which are often either typos or project-specific terms with limited generalizability. Second, we excluded overly common words found in more than 80% of the documents, which offer little discriminatory power. After applying these filters, the resulting vocabulary was reduced to 2,098 distinct tokens. This refined feature space significantly reduced memory usage while preserving the core semantic structures needed for clustering and downstream modeling.

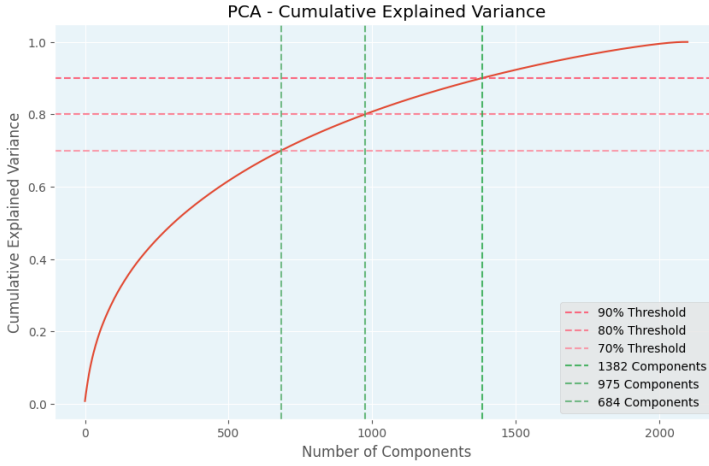


Figure 5.1: Cumulative Variance Retention in PCA at Key Thresholds (90%, 80%, 70%)

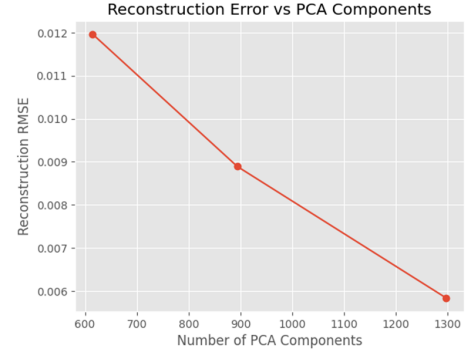


Figure 5.2: RMSE Trend Across Principal Components

Dimensionality Reduction via PCA

After generating the final TF-IDF matrix, we used PCA to reduce its dimensionality while keeping the most important information. To determine the number of components to keep, we examined the cumulative explained variance. As shown in Figure 5.1, retaining 975 principal components can capture about 80% of the original variance. This provided a solid balance between data representation and computational efficiency.

To quantitatively assess the fidelity of this reduced representation, we computed the RMSE between the original and reconstructed TF-IDF matrices. RMSE serves as an error metric for information loss due to dimensionality reduction. As shown in Figure 5.2, the RMSE dropped notably from 0.012 at 684 components to 0.0088 at 975 components. Beyond this point, further reductions in error became marginal, indicating diminishing returns. This inflection point represents an effective "elbow" in the trade-off between accuracy and compression.

Based on our analysis, we selected 975 principal components to represent the project descriptions. These compressed vectors served as effective inputs to the unsupervised clustering models, allowing them to detect meaningful similarities and structure within the projects.

While TF-IDF effectively highlights term frequency weighted by document rarity, it treats descriptions as bag-of-words mixtures of latent topics, neglecting word semantics and contextual dependencies. To address this limitation, we adopted BERT and BERTopic to extract more coherent and semantically rich topic representations.

5.2.2 Semantic Clustering via Pre-trained Language Models: BERT

The second pipeline leverages a deep contextual language model, BERT, to generate semantically rich embeddings of project descriptions. We adopt a feature-based strategy, using the `bert-base-uncased` variant (Devlin et al., 2018)¹, which is trained on lowercase English text without fine-tuning its weights. Input text is first tokenized using the `BertTokenizer` provided in Hugging Face's transformers (Wolf et al., 2020) library. Each description is tokenized to a fixed length

¹<https://huggingface.co/google-bert/bert-base-uncased>

(maximum 128 tokens), mapped to corresponding vocabulary indices, and then passed into the BERT model. The model outputs contextualized hidden representations for each token. From these, we extract the embedding of the special classification token [CLS], which encapsulates the overall semantic content of the input sequence. The resulting embedding is a 768-dimensional vector and serves as the input for downstream tasks, including clustering, classification, and direct integration into a ranking model.

5.2.3 Topic Modeling for Project Grouping via BERTopic

In the third pipeline, we adopt BERTopic to group projects based on latent topics extracted from their textual descriptions. BERTopic is particularly well-suited for capturing semantic patterns in short texts, making it effective for modeling GitHub project descriptions.

We evaluate two configurations with different parameters:

- **Custom BERTopic Pipeline:** This configuration leverages the `all-MiniLM-L6-v2` model from SentenceTransformers. We experiment with different minimal numbers of samples in the cluster: 5, 15, and 30, respectively, and set the number of neighbors in UMAP to 15. This setup allows precise tuning of clustering resolution and topic coherence.
- **Default BERTopic Configuration:** `distiluse-base-multilingual-cased-v2` model is used in this approach, which offers quick deployment support. Although it provides less customization, it is computationally efficient and ideal for scenarios requiring rapid experimentation or support for diverse languages.

5.2.4 Clustering and Performance Evaluation

We implemented four clustering methods on project representations: K-Means, Hierarchical Clustering, DBSCAN, and HDBSCAN. To compare the effectiveness of different project clustering strategies for better recommendation system performance, we evaluate the clustering quality using three widely adopted metrics: the Silhouette Coefficient, the Calinski-Harabasz Index, and the Davies-Bouldin Index.

5.2.4.1 K-Means Clustering

As shown in Figure 5.3, graph (a) illustrates that the Silhouette Coefficient reaches a noticeable peak when the number of clusters is around 45. A closer examination in graph (b) reveals that the highest silhouette score of 0.6474 is achieved when using 47 clusters. Therefore, we select the optimal number of clusters as 47 for the TF-IDF representation with K-Means. For the second method, we apply K-Means clustering to BERT-based embeddings of project descriptions. As shown in Figure 5.4, graph (a) presents the Silhouette Coefficient when the number of clusters ranges from 2 to 10. The results show a steady decline in silhouette scores, dropping from 0.6267 at 2 clusters to 0.5389 at 10 clusters. Graph (b) expands the range from 2 to 100 clusters and reveals that the Silhouette Coefficient continues to decrease sharply before gradually converging around the range of 0.51 to 0.52 as the number of clusters increases. This pattern indicates that the cluster quality deteriorates as more groups are formed, with diminishing gains in structure or separability.

5.2.4.2 Hierarchical Clustering

Figure 5.5 presents the evaluation results of hierarchical clustering applied to TF-IDF-based project representations. In graph (a), the Silhouette Coefficient peaks near a cluster count of 20, indicating an initial estimate of optimal segmentation. Upon narrowing the range from 3 to 30 clusters, subfigure (b)

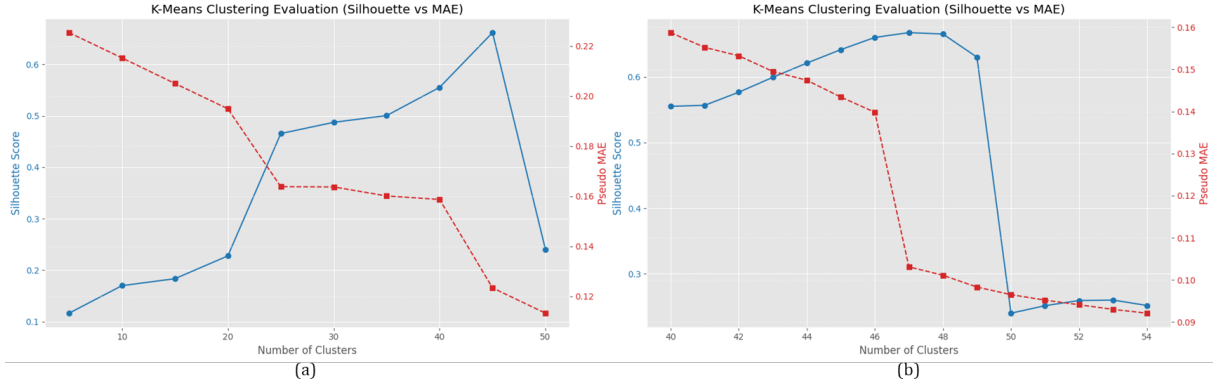


Figure 5.3: Silhouette Scores for k-means Clustering with TF-IDF tokenized project descriptions

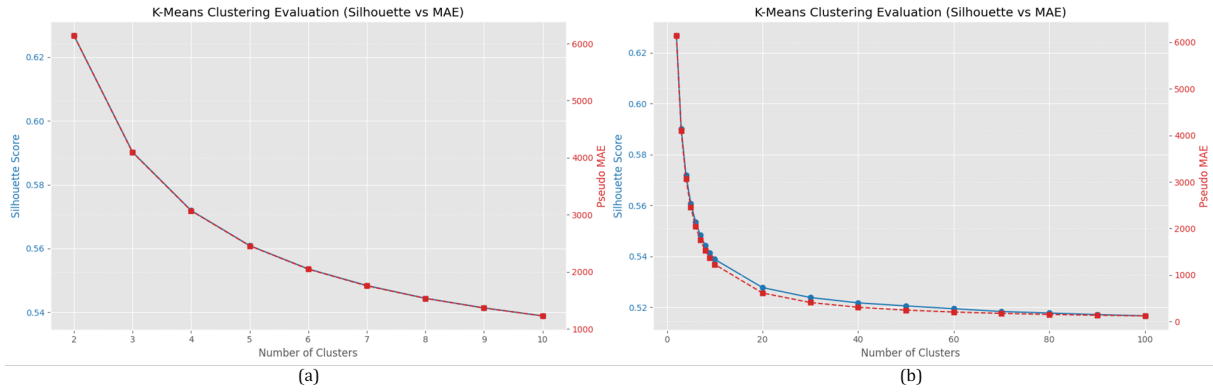


Figure 5.4: Silhouette Scores for k-means Clustering with BERT-Based tokenized project descriptions

reveals that the highest Silhouette Coefficient of 0.3819 is achieved at 18 clusters, which we adopt as the representative configuration for the TF-IDF setup.

Next, we evaluate hierarchical clustering using BERT-based embeddings (Figure 5.6). Subfigure (a) displays substantial variation in Silhouette Scores when varying the number of clusters from 2 to 10, suggesting sensitivity to cluster granularity. A focused view in subfigure (b), spanning 2 to 15 clusters, shows that a 5-cluster configuration yields the highest Silhouette Coefficient of 0.5457. Given its superior internal validity and semantic coherence, the BERT embedding with five clusters is selected for further comparison with alternative clustering methods.

5.2.4.3 DBSCAN Clustering Analysis

In the DBSCAN experiment using TF-IDF features, the number of clusters was automatically determined by the algorithm. We set the minimal number of samples in the cluster (`min_samples`) to 15 and varied the ϵ value across a range from 0.3 to 2.0 in 15 evenly spaced steps. As shown in Figure 5.7, the highest Silhouette Coefficient was achieved at $\epsilon = 0.786$, with a value of only 0.0236. Furthermore, a significant portion of the data points was labeled as noise (with 8750 noise points), indicating that DBSCAN struggled to form meaningful or coherent clusters under the TF-IDF representation. These results suggest that TF-IDF features are not well-suited for DBSCAN in this context.

When we switched to BERT-based embeddings for project clustering, DBSCAN's performance improved dramatically. As shown in Figure 5.8, the highest Silhouette Coefficient reached 0.9369 when using 8

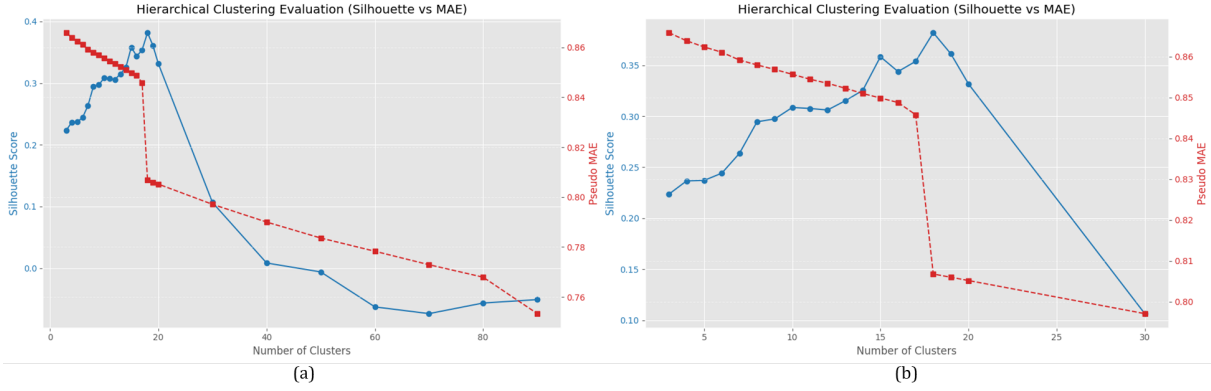


Figure 5.5: Silhouette Scores for Hierarchical Clustering with TF-IDF tokenized project descriptions

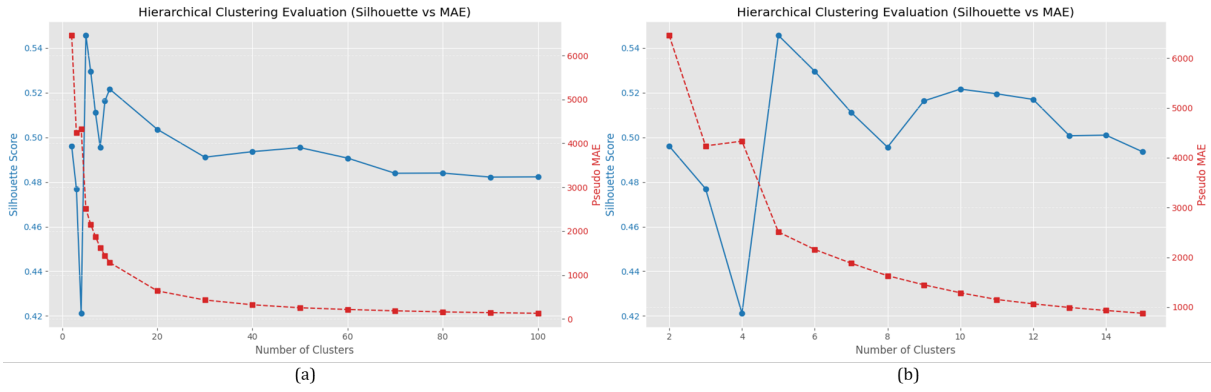


Figure 5.6: Silhouette Scores for Hierarchical Clustering with BERT-Based tokenized project descriptions

clusters—suggesting a highly coherent and well-separated grouping. This result indicates that 8 is a reasonable and ideal number of clusters under BERT-based embeddings. In addition to the high Silhouette Score, this configuration also achieved the lowest Davies-Bouldin Index and the highest Calinski-Harabasz score among all tested methods.

Overall, DBSCAN with BERT-based embeddings significantly outperformed its TF-IDF counterpart. Its clustering quality, supported by all three evaluation metrics, demonstrates that this combination is highly effective for uncovering meaningful latent structure in project descriptions and programming language features. These properties make it particularly well-suited for downstream tasks such as project recommendation systems.

5.2.4.4 HDBSCAN

Using HDBSCAN as the clustering method on TF-IDF-tokenized project descriptions, we first applied UMAP for dimensionality reduction with the number of neighbors set to 15 and a minimum distance of 0.1. The clustering process resulted in 52 clusters, as shown in Figure 5.9. The data is present in Table 5.2, the highest Silhouette Coefficient achieved was 0.1808. Compared to the performance of K-Means and Hierarchical Clustering, this result is considerably lower, suggesting that HDBSCAN was less effective in identifying well-separated and cohesive clusters under the TF-IDF representation.

Then we experimented with a BERT-based model to tokenize project descriptions, and the resulting cluster distribution is visualized in Figure 5.10. The model ultimately identified 46 clusters. However, the

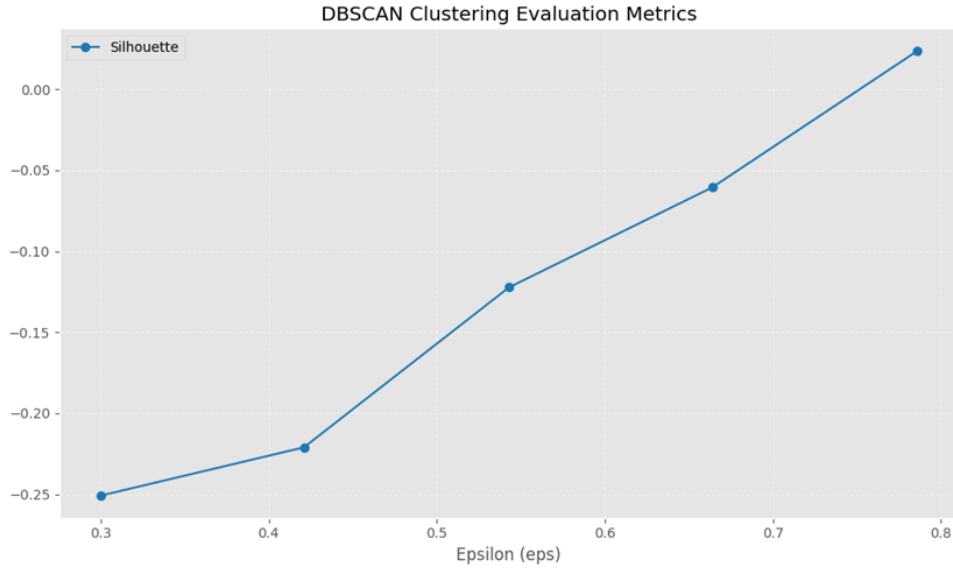


Figure 5.7: Silhouette Scores for DBSCAN Clustering with TF-IDF tokenized project descriptions

scatter plot reveals noticeable overlap between clusters, and the inter-cluster distances are not clearly separable. To evaluate the clustering performance, we applied five different parameter settings and assessed the results using three internal metrics: SI, CHI, and DBI, as shown in Table 5.3.

When setting the minimum cluster size to 30, we consistently obtained the same silhouette score of 0.3269, regardless of increases in the number of neighbors or minimum distance parameters. These adjustments also didn't have any impact on CHI and DBI values, indicating limited sensitivity to these hyperparameters. We also increase the minimum cluster size to 50. Under this configuration, although the silhouette score rises up to 0.3859, the model produced only 4 clusters, accompanied by a substantial amount of noise data, which suggests poor clustering compactness and separation.

BERTopic, by default, employs HDBSCAN as its clustering algorithm; the results are presented in Table 5.4. However, across two transformer configurations and parameter settings tested on the dataset of 49,129 GitHub projects, we consistently observed that over 20,000 data points were labeled as noise. Additionally, the highest Silhouette Score achieved was only 0.0711, indicating poor cluster separation. Despite these limitations, we still proceed with using BERTopic (Method 3 in Table 5.4 for building the recommendation system, to test whether it can perform better than the baseline.

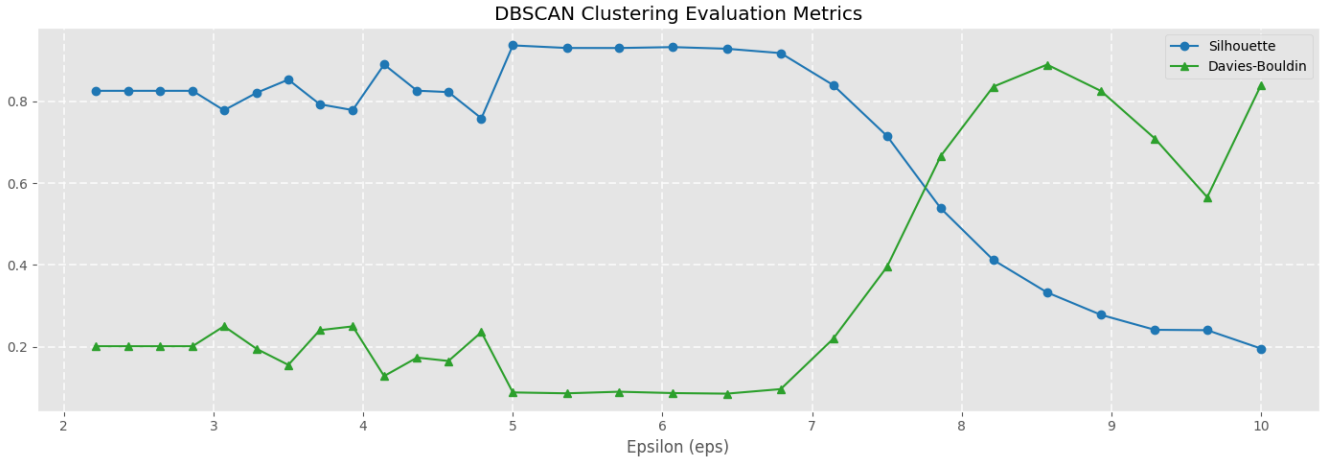


Figure 5.8: Silhouette Scores and Davies-Bouldin Score for DBSCAN Clustering with BERT-Based tokenized project descriptions

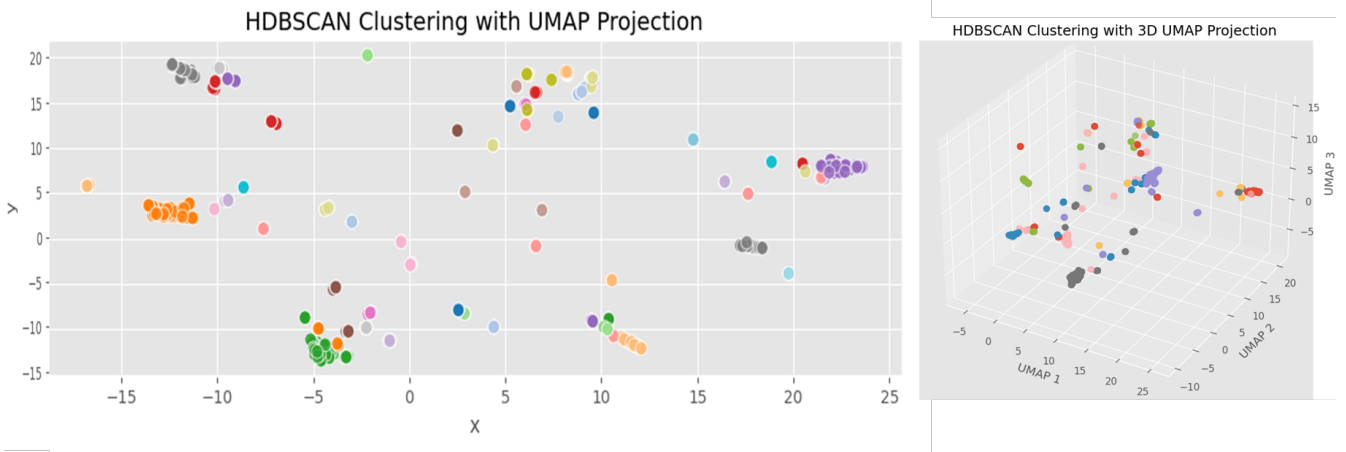


Figure 5.9: HDBSCAN as the clustering method with TF-IDF tokenized project descriptions

Results of Clustering Techniques

Because TF-IDF and BERT embeddings are fundamentally different in how they represent text, TF-IDF is sparse and keyword-driven, while BERT captures dense semantic relationships. The results of clustering evaluation metrics (SI, CHI, DBI) vary significantly between the two. Therefore, these scores should not be directly compared across representation methods. Instead, they serve as internal benchmarks that reflect how well clusters are formed within each specific embedding space. In our final setup, we selected three distinct combinations: TF-IDF with K-Means clustering (yielding 47 clusters), BERT embeddings with DBSCAN clustering (producing 8 clusters), and BERTopic integrated with our custom model (resulting in 230 clusters). Each clustering output serves as input to the recommendation system pipeline, and the performance of these models will be evaluated and compared in Chapter 6.

5.3 Candidate Generation: Neural Collaborative Filtering

To generate candidate recommendations, we employ a NCF model to predict the likelihood of seven types of interactions between developers and OSS projects. The complete structure of the NCF model is illustrated in Figure 5.11. Each input sample is represented by a concatenated feature vector comprising

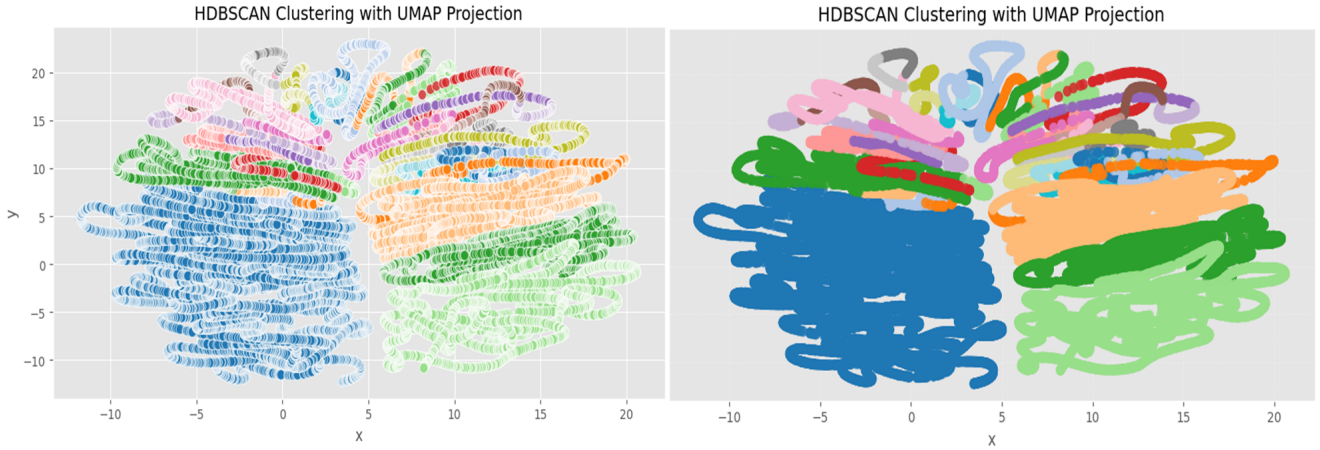


Figure 5.10: HDBSCAN clustering results using UMAP projection of BERT-based tokenized project descriptions. A total of 46 clusters were identified. The left plot displays data points with white edges, while the right plot omits edges for a cleaner contrast.

TF-IDF + PCA	#Cluster	Davies-Bouldin Index	Calinski-Harabasz Score	Silhouette Score
K-means	47	0.9690	28.2601	0.6474
K-means	522	1.4632	15.2942	0.2294
Hierarchical Clustering	18	1.2344	18.5077	0.3819
DBSCAN (Eps = 0.786)	5	13.7897	16.1000	0.0236
HDBSCAN	52	2.1008	9.0208	0.1808

Table 5.1: Comparison of Clustering Algorithms Using Unsupervised Metrics with TF-IDF and PCA for Topic Modeling

BERT	#Cluster	Davies-Bouldin Index	Calinski-Harabasz Score	Silhouette Score
K-means	2	0.5000	147367.52	0.6267
Hierarchical Clustering	5	0.5000	276131.16	0.5457
DBSCAN (Eps = 5.00)	8	0.0883	51361618.38	0.9369
HDBSCAN	46	0.4849	14422.10	0.3269

Table 5.2: Comparison of Clustering Algorithms Using Unsupervised Metrics on BERT-Based tokenized project descriptions

HDBSCAN	Method 1	Method 2	Method 3	Method 4	Method 5
Min_sample	15	30	30	30	50
N_neighbor	15	15	50	15	15
Min_dist	0.1	0.1	0.1	0.3	0.1
# Cluster	215	46	46	46	4
Silhouette Score	0.3184	0.3269	0.3269	0.3269	0.3859
CH Index	14023.93	14422.10	14422.10	14422.10	37591.76
DB Index	0.4793	0.4849	0.4849	0.4849	0.4959

Table 5.3: Comparison of HDBSCAN Clustering Performance under Different Parameters on BERT-Based Project Description

BERTopic	Model 1 (Default)	Model 2	Model 3
Transformers	Distiluse-base-multilingual-cased-v2	All-MiniLM-L6-v2	
Min_sample	5	15	30
N_neighbor	15	15	15
Min_dist	0.1	0.1	0.1
# Cluster	577	463	230
Silhouette Score	0.0051	0.0599	0.0711
Calinski-Harabasz Score	18.63	35.91	44.03
Davies-Bouldin Index	3.7333	2.6323	2.5458
Noise	20713	20020	21768

Table 5.4: Comparison of BERTopic with HDBSCAN Using Different Transformer Embeddings and Clustering Parameters Evaluated by Unsupervised Metrics

four categorical features (programming language, location, country, and company), three normalized continuous features (technical ability, profile similarity, and social tie), a clustering feature derived from project descriptions, and identifiers for both the developer and the project. These features collectively capture individual characteristics as well as relational dynamics.

Model Architecture

The input vector z_0 , formed by concatenating all feature representations, is passed through a Multi-Layer Perceptron (MLP) with two fully connected hidden layers. Each layer uses the ReLU activation function to introduce non-linearity. The network structure follows: 128 ReLU $\rightarrow$ 64 ReLU.

The transformation process is formally defined as:²

$$z_1 = \text{ReLU}(W_1 z_0 + b_1) \quad (5.1)$$

$$z_2 = \text{ReLU}(W_2 z_1 + b_2) \quad (5.2)$$

$$\hat{y} = W_3 z_2 + b_3 \quad (5.3)$$

The model will be trained until convergence overall GitHub developers, corresponding to several epochs over the data. The final output is a 7-dimensional logit vector $\hat{y} \in \mathbb{R}^7$, with each dimension corresponding

² W_1 , W_2 , and W_3 are the learnable weight matrices. b_1 , b_2 , and b_3 are the corresponding biases.

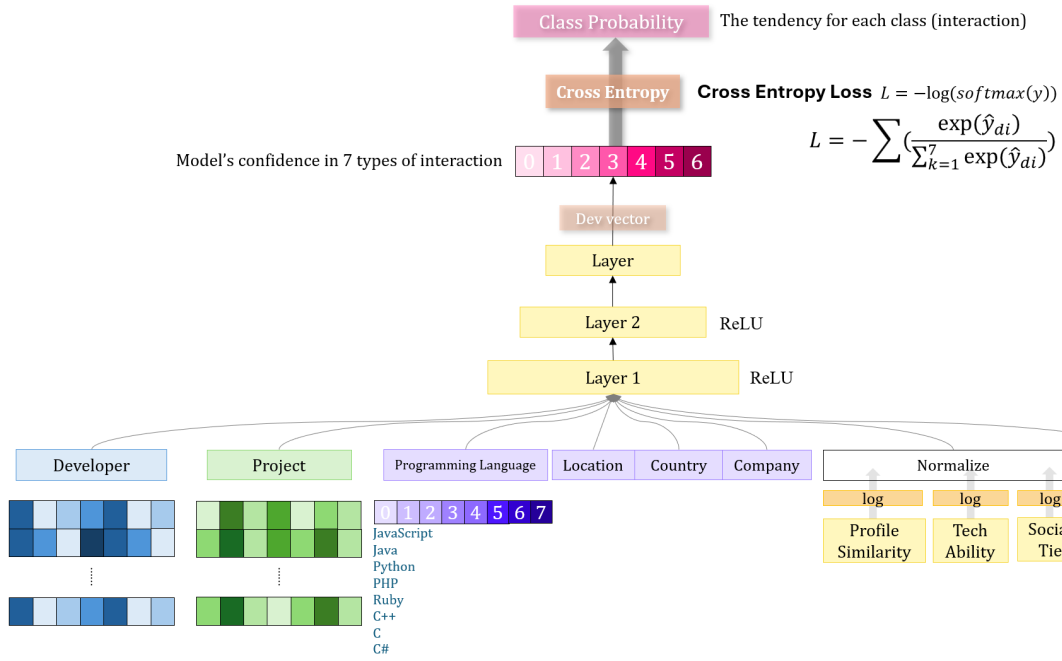


Figure 5.11: Architecture of the NCF-based candidate generation model, showing embedded sparse features concatenated with continuous features. Deep candidate generation model architecture showing embedded sparse features concatenated with dense features.

to one of the predefined interaction types.

The model is trained until convergence over all GitHub developers, typically across several epochs. The final output is a 7-dimensional logit vector $\hat{y} \in \mathbb{R}^7$, where each dimension corresponds to one of the predefined interaction types. During inference, the softmax function is applied to $\hat{y}$ to obtain the probability distribution over these interaction types.

Output, Loss Function and Training Objective

The output layer of the model consists of 7 neurons, each corresponding to one interaction type. We applied the softmax activation function to obtain a probability distribution over the 7 classes and trained the model using the cross-entropy loss, which measures the difference between the predicted probability distribution and the true interaction class. The overall loss across all training samples is:

$$L = -\sum_{d,i} \log\left(\frac{\exp(y_{di}^{(c)})}{\sum_{k=1}^7 \exp(y_{di}^{(k)})}\right) \quad (5.4)$$

This formulation ensures that the model learns to assign high probability to the correct interaction type while minimizing the likelihood of incorrect types. It allows for an understanding of developer behavior beyond binary relevance, making the recommendation system more expressive and personalized.

Finally, we construct a new training dataset for the ranking model by generating 300 candidate projects for each of the 49,125 target projects. The resulting dataset combines these candidate lists into a comprehensive collection tailored for learning to rank.

5.4 Ranking: Learning-to-Rank (LETOR) with XGBoost

To improve the quality of recommendations generated during the candidate generation stage, we apply a LETOR approach using XGBoost. At this stage, the candidate list is re-ranked based on more precise textual descriptions and the logit vectors produced by the NCF model. This refinement prioritizes projects with a higher predicted likelihood of meaningful developer interaction.

Feature Design and Model Architecture

The ranking model uses the same 9 input features as the NCF model, and adds 2 meaningful features.

1. **Project Description:** Instead of using cluster embeddings, we use the full 768-dimensional BERT embedding of the project description to capture richer semantic information.
2. **NCF Logits:** We incorporate the 7-dimensional logit output vector from the first-stage NCF model, which represents the model's confidence scores for each interaction type. These logits provide valuable prior knowledge that guides the ranking model.

The final feature vector includes high-level semantic content and learned behavior patterns from the NCF model will be passed into XGBoost and re-ranked by the ranking objective function: NDCG.

By directly aligning the loss function with our evaluation metric, this objective helps produce more meaningful recommendation lists, where the most likely interaction candidates appear earlier on the list for each developer.

Training Strategy

Each developer in the dataset is treated as an independent query, with their corresponding candidate projects forming a training group. For every developer–project pair, a feature vector is constructed based on the full model input. To train the model, we group the data using a `group` array that specifies the number of candidate projects per developer query. This allows the learning-to-rank with XGBoost model to rank items within each group independently.

We employed the model with the following hyperparameters: a learning rate of 0.05 to control the contribution of each tree, a maximum depth of 6 to reduce overfitting, 300 boosting rounds (`n_estimators`) to allow sufficient model capacity, and the histogram-based tree construction method (`tree_method="hist"`) for computational efficiency. Each developer was treated as a separate ranking group, with relevance labels derived from observed interaction ratings (ranging from 0 to 6), enabling the model to learn nuanced preferences from both semantic signals and behavioral patterns.

After training, the model assigns a relevance score to each candidate project, which is then used to re-rank the list for each developer. For evaluation, we use NDCG@K to assess the quality of the ranked results, and Recall@K to measure the success rate of retrieving relevant recommendations.

5.5 Baseline Selection

To ensure a rigorous evaluation of our recommendation approach, we establish five baselines that are discussed in Section 3, both personalized and non-personalized, to evaluate whether our recommendation model can accurately predict projects that align with specific developer interests. The goal is to surpass naive heuristics. These baselines are widely used and easy to interpret. All five models use BERT embedding to represent the description and the other 9 features. The inclusion of these models enables us to validate that our method offers meaningful enhancements over standard recommendation strategies.

Chapter 6

Evaluation and Result

In this chapter, we evaluate the performance of our recommendation system.

6.1 Constructing the ground truth

As outlined in Chapter 4, the validation period spans from January 1, 2016, to December 31, 2018. In the training set, we include a total of 49,125 projects, representing all candidate repositories observed during the training period. For consistency in evaluation, the validation set consists of 47,198 projects that also appear in the training set, thus avoiding cold start scenarios and ensuring that the system does not attempt to recommend unseen items. The validation data includes a total of 2,524,743 interaction records, which capture developer engagement with these projects during the evaluation period.

To ensure the fairness and validity of the evaluation, the validation set is restricted to developers and projects present in the training data. This design decision prevents the recommendation system from attempting to suggest projects it has never seen before, avoiding out-of-distribution issues while establishing a controlled benchmark for relevance modeling. However, this constraint also introduces a natural performance ceiling, as the system cannot recommend entirely new or recently created projects.

6.2 Evaluation of the Recommendation System

In this section, we evaluate the effectiveness of our proposed recommendation system by integrating three distinct content-based clustering strategies—TF-IDF, BERT embeddings, and BERTopic—into both the candidate generation and ranking pipeline. Each clustering configuration is assessed in comparison with five baseline models to address Research Question 1. Evaluation is performed using standard ranking metrics, including Recall@K and NDCG@K, which capture both the coverage completeness and the quality of item ordering. These metrics provide a comprehensive assessment of Top-K recommendation performance and are well-suited to the goals of delivering personalized and well-ranked suggestions.

6.2.1 One-stage Recommendation System

In the first stage of our recommendation pipeline, we apply an NCF model embedding with all available features. To evaluate the model’s ability to retrieve relevant projects, we generate top- k recommendation candidates with different values of k . Specifically $k = 10, 50, 100$, and 300 . We then compute the corresponding Recall@ k scores, as shown in Table 6.1.

We expected and observed that when the number of candidates generated increases, the recall score increases. Among all clustering-based variants, the model that leverages project description embeddings generated by BERT and applies DBSCAN clustering with 8 groups achieves the highest recall score, reaching Recall@300 of 0.45033. This configuration consistently outperforms alternative clustering approaches across all evaluated k values, for instance, TF-IDF achieves Recall@300 of 0.31391, while BERTopic produces 0.18262. We observe a substantial performance gain between Recall@10 and

Recall@50 across all three clustering methods, nearly doubling the recall values. Furthermore, when generating 100 candidates, the BERT-based clustering model achieves $\text{Recall@100} = 0.30035$, which is comparable to TF-IDF’s performance at $k = 300$, underscoring the efficiency of complete semantic embeddings in candidate generation.

The NDCG index is illustrated in Table 6.2. We observe that as the number of recommended candidates increases, the NDCG score also increases slightly. However, this rise is relatively modest compared to the improvement seen in Recall@K. All NDCG scores in our experiments remained below 0.1, indicating suboptimal ranking quality across configurations. Among all tested setups, the highest score was achieved using BERT embeddings combined with DBSCAN clustering, reaching NDCG@300 of 0.05194. This configuration outperformed the TF-IDF model and the baseline XGBoost model, though none demonstrated strong positional precision. Notably, the BERTopic-enhanced candidate lists underperformed relative to BERT and TF-IDF, despite the addition of semantic topic representations.

Given the low NDCG scores across all results, it is clear that the current model struggles with how to effectively learn fine-grained ranking preferences. These results suggest the need for a more effective ranking strategy, potentially involving advanced and richer feature representations and more expressive learning-to-rank architectures.

6.2.2 Baseline Performance Comparison

Among the five baseline models, XGBoost and Project-based k NN are the two outperforming models, followed closely by Developer-based k NN. At Recall@10, both XGBoost (0.01834) and Project-based k NN (0.01832) demonstrate stronger performance than our BERTopic-based recommendation system (0.01639), showing their competitive strength in early ranking. In contrast, the Random and Top-10 Popular baselines perform poorly. They barely identify any relevant projects within the top-10 recommendations, resulting in Recall@10 scores near zero. Even when expanding the candidate pool to 300 (Recall@300), their performance remains marginal with only 0.00111 and 0.00034, respectively. This underscores the critical importance of a well-designed recommendation system that can effectively capture developer preferences and behavioral signals, rather than relying on random or popularity-based assumptions.

Notably, Project-based k NN consistently outperforms Developer-based k NN baseline, from Recall@10 to Recall@300 (specifically: 133%, 155%, 18%, 9%), suggesting that project characteristics still play a central role in attracting developers. This insight emphasizes the importance of accurately capturing and embedding project description semantics. Selecting effective tokenization and embedding strategies for representing projects is essential for improving overall recommendation quality.

6.2.3 Two-Stage Recommendation System

The overall Recall@K results for the two-stage recommendation pipeline are summarized in Figure 6.3a. The experimental findings indicate a substantial improvement in recommendation quality after the introduction of the ranking stage. Among all tested configurations, the BERT-DBSCAN method continues to achieve the highest performance, reaching Recall@10 of 0.21193. This suggests that, on average, each developer receives approximately two relevant project recommendations within the top 10 results. The outcome highlights the system’s ability to deliver targeted and useful suggestions with minimal browsing effort. In practical terms, developers are able to identify projects aligned with their interests or expertise

after reviewing only a small candidate set, demonstrating the model’s strength in surfacing meaningful interactions early in the ranked list. In comparison, the candidate generation model alone (without the ranking component) yields a significantly lower Recall@10 of 0.08622. Thus, the full BERT-DBSCAN pipeline improves early-stage recall by approximately 2.5 times, clearly demonstrating the impact of the ranking stage. NDCG further validates our ranking quality. Table 6.3b reports the NDCG@10 and NDCG@50 scores across all model configurations. Our best-performing model “BERT with DBSCAN clustering” achieves $\text{NDCG@10} = 0.08003$ and $\text{NDCG@50} = 0.12304$. These results confirm that the top-ranked projects are not only relevant but also properly ordered by their importance to the user.

To further evaluate ranking efficiency, we compare the number of candidates required to achieve a target recall score. Using the BERT-based candidate generator alone, we must generate 100 candidates to reach $\text{Recall@100} = 0.30035$. However, with the addition of the XGBoost ranking model, we achieve a comparable score ($\text{Recall@50} = 0.30082$) using only 50 candidates. This highlights the efficiency gain introduced by the ranking stage, enabling the system to surface relevant results more effectively with fewer candidates.

When comparing the three clustering-based variants used in our two-stage recommendation system pipeline, the final performance consistently follows the ranking:

$$\text{BERT} > \text{TF-IDF} > \text{BERTopic}$$

This order remains stable across all tested values of k , indicating robustness across different recall and NDCG evaluation cutoffs.

6.2.4 Training Time Discussion

We compared the processing time of three project description encoding methods and found substantial differences in computational cost and downstream effectiveness.

The TF-IDF tokenizer is the most efficient. Despite requiring additional preprocessing, the entire process, including dimensionality reduction, took only 248.5 seconds. This makes it a practical choice for large-scale applications when speed is a priority. In contrast, the BERT-based encoder is significantly more computationally expensive. Encoding all project descriptions required approximately 643 minutes, due to the model’s deep architecture and the need to compute contextual embeddings for each token sequence. However, this method provides rich semantic representations, which contribute significantly to model accuracy in ranking tasks.

The BERTopic method completed its full pipeline in about 235 minutes. While it is faster than BERT, BERTopic’s final performance often fell short of the simple XGBoost baseline, which required only 347 seconds to train. Although BERTopic offers a reasonable trade-off between computational cost and representational power, its relatively low ranking accuracy suggests it is not suitable as a final model in the GitHub context. Future work may consider using BERTopic’s topic distributions as auxiliary features rather than primary inputs and combining them with BERT embeddings in a hybrid architecture.

The significant speed advantage of the XGBoost baseline, along with its strong performance, further justifies our adoption of LETOR with XGBoost Ranker in the second stage of our recommendation system, where both efficiency and precision are critical. Our final approach combines the BERT embedding NCF

model with the XGBoost ranker to score and order candidate projects. This hybrid strategy enables us to deliver high-quality recommendations while maintaining reasonable computational costs.

Model		Recall			
		@10	@50	@100	@300
NCF Model	TFIDF-Kmeans	0.02931	0.05130	0.14657	0.31391
	BERT-DBSCAN	0.08622	0.17364	0.30035	0.45033
	BERTopic	0.01639	0.03037	0.08262	0.18262
	XGBoost	0.01834	0.02463	0.07556	0.11986
Developer-based kNN		0.00957	0.01082	0.06866	0.09644
Project-Based kNN		0.01832	0.02769	0.08146	0.10592
Random		0.00036	0.00051	0.00086	0.00111
Top 10 Popular		0.00006	0.00018	0.00033	0.00034

Table 6.1: Recall@K Accuracy Comparison for Generated Models(NCF model) and Benchmark Methods

Model		NDCG			
		@10	@50	@100	@300
NCF Model	TFIDF-Kmeans	0.03372	0.03547	0.03985	0.04925
	BERT-DBSCAN	0.03792	0.03910	0.04893	0.05194
	BERTopic	0.02224	0.02688	0.02892	0.03352
	XGBoost	0.02447	0.02740	0.02943	0.03422
Developer-based kNN		0.01933	0.02469	0.02736	0.03104
Project-Based kNN		0.01486	0.01744	0.02536	0.03073
Random		0.00067	0.00082	0.00109	0.00198
Top 10 Popular		0.00000	0.00000	0.00108	0.00133

Table 6.2: NDCG@K Accuracy Comparison for Generated Models(NCF model) and Benchmark Methods

Model		Recall @10	Recall @ 50
NCF + XGBoost	TFIDF-Kmeans	0.17591	0.26793
	BERT-DBSCAN	0.21193	0.30082
	BERTopic	0.08294	0.18533
	XGBoost	0.01834	0.02463
Developer-based kNN		0.00957	0.01082
Project-Based kNN		0.01832	0.02769
Random		0.00036	0.00051
Top 10 Popular		0.00006	0.00018

(a) Recall Results

Model		NDCG @10	NDCG @ 50
NCF + XGBoost	TFIDF-Kmeans	0.06294	0.08522
	BERT-DBSCAN	0.08003	0.12304
	BERTopic	0.04982	0.05331
	XGBoost	0.02447	0.02740
Developer-based kNN		0.01933	0.02469
Project-Based kNN		0.01486	0.01744
Random		0.00067	0.00082
Top 10 Popular		0.00000	0.00000

(b) NDCG Results

Table 6.3: Accuracy Comparison of Proposed Two-Stage Models (NCF with XGBoost) and Benchmark Methods Across (a) Recall@K and (b) NDCG@K

6.2.5 XGboost Feature Importance

We compute gain-based feature importance from the LambdaMART-style ranking model implemented in XGBoost. In this analysis, *gain* refers to the average improvement in the model’s objective function (NDCG) brought by splits using a given feature. The result is illustrated in Figure 6.1, revealing a distinct hierarchy in how features contribute to relevance prediction. Among all input variables, technical ability, profile similarity, and social tie score are the top three contributors, with gain scores of 71.38, 62.15, and 43.35, respectively. These features capture latent relationships between developers and projects by modeling programming knowledge, similarity in profile attributes, and social connections. This suggests that personal behavioral signals provide stronger discriminative power than static metadata. In contrast, categorical identifiers such as item, user, and metadata attributes (location, company, language, country) yield substantially lower gains. While these features help better understand developers and assign projects to them, their limited variability and discrete encoding restrict their expressiveness in learning fine-grained relevance patterns. The relatively modest gain scores suggest that static metadata alone is insufficient for modeling personalized interaction likelihoods.

The model also incorporates seven logit scores from the upstream NCF model, each corresponding to a type of interaction behavior. These behavioral scores further enrich the feature space with collaborative filtering signals. Among them, the logit corresponding to ‘Commit’ interactions has the highest gain, followed by Pull Request Commit, Pull Request, and Issue. Star and Fork exhibit the lowest gain scores. This ordering aligns with the depth of engagement associated with each interaction type, suggesting that developers are more predictable and more meaningfully ranked when they demonstrate intent to contribute substantively. In contrast, lightweight interactions like starring or forking are less influenced by personal or behavioral features and contribute less to ranking performance. This result confirms that when the NCF model predicts a developer is likely to engage through high-interest behaviors such as commits or Pull Request, the ranking model places greater emphasis on those candidates. Given the overall high recall achieved, these interaction-type logits serve as effective behavioral intent indicators, enabling the system to prioritize developer-project matches with deeper engagement potential.

Additionally, each BERT-based semantic embedding of project descriptions consists of 768 dimensions. We aggregate the gain scores across all dimensions, resulting in an average total importance of 46.74, ranking as the third most influential feature in our model. This highlights the value of contextual semantic understanding, enabling the system to interpret topical relevance between developer interests and project content, thereby improving the quality of recommendations.

Taken together, this feature importance analysis highlights the central role of skill alignment, network proximity, and semantic content in personalized ranking. It reinforces the effectiveness of a hybrid strategy that integrates deep contextual embeddings, collaborative behavioral signals, and structured interaction features. We note, however, that gain-based importance can be biased toward high-cardinality features, and interpret these results as indicative rather than causal.

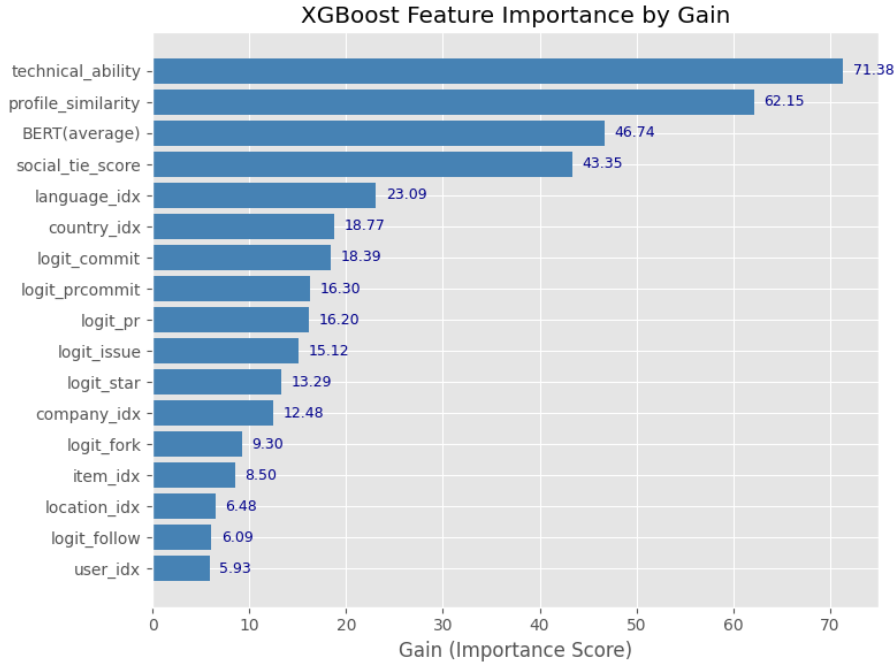


Figure 6.1: XGBoost Feature Importance

6.3 Research Question Result

In this section, we present the answers to our research questions (*QR*).

RQ 1: To what extent can the proposed two-stage recommendation system accurately recommend open-source projects for developers?

The choice of using BERT embeddings to fully capture the semantic meaning of each project description and apply DBSCAN clustering to group similar projects consistently outperformed other representations and clustering methods, demonstrating the clear advantage of deep semantic representations over token-based approaches. Furthermore, our results show that ranking is a necessary component of an effective recommendation pipeline. Incorporating a learning-to-rank model significantly improves both recall and ranking quality (NDCG). Together, these enhancements address the core goals of personalization and ranking in our research question, confirming the system’s capability to deliver practical, relevant, and high-quality developer-project recommendations.

RQ 2: What key features influence a developer’s choice of an open-source project, and how can they be effectively modeled?

Analysis of feature importance from the LambdaMART (XGBoost) model reveals that behavioral features are far more influential than static metadata in driving recommendation quality. The top-ranked features: Technical Ability, Profile Similarity, and BERT-based project embeddings, followed by Social Ties. These features suggest that a developer’s skill set and network, alignment with the existing contributor profile, and social proximity to the project team are strong predictors of participation. Notably, BERT embeddings effectively capture semantic information from project descriptions, bridging the gap between textual content and developer interests.

RQ 3: How can deep learning on structured data, including Transformer-based NLP and XGBoost models, enhance recommendation accuracy?

This research demonstrates the effectiveness of combining Transformer-based NLP with LETOR in XGBoost to improve recommendation performance. Specifically, we integrate BERT embeddings to capture semantic representations of project descriptions, thereby enriching the candidate generation process with deep contextual understanding. The tree-based ranking model is capable of capturing nonlinear interactions and feature hierarchies, making it well-suited for interpreting complex user–item relationships. Experimental results show that this hybrid architecture significantly boosts Top-K recommendation quality, outperforming traditional baselines in both Recall@K and NDCG metrics. These findings highlight the complementary strengths of deep NLP models and structured ranking algorithms, supporting a unified design paradigm where semantic richness and behavioral relevance jointly drive personalized recommendation accuracy.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

In this paper, we presented a two-stage recommendation system for the open-source software (OSS) projects platform: GitHub, trained on GHTorrent data from 2010 to 2015. Our framework decomposes the recommendation task into two core components: candidate generation and ranking.

For candidate generation, we employed Neural Collaborative Filtering (NCF) as the foundational model, enriched with a diverse feature set comprising four categorical attributes (programming language, location, country, and company), three normalized continuous variables (technical ability, profile similarity, and social ties), a clustering-based topic feature derived from project descriptions, and identifiers for both developers and projects. The multi-layer architecture effectively captured non-linear interactions among these heterogeneous features.

To improve representation learning from project descriptions, we systematically evaluated three text embedding techniques and four clustering algorithms. Our experiments demonstrated that BERT embeddings, when combined with DBSCAN clustering, consistently outperformed traditional approaches such as TF-IDF and BERTopic. Despite being computationally intensive, the BERT-based with DBSCAN clustering approach achieved the highest Silhouette Score of 0.9384, indicating superior cluster quality and delivered the best downstream Recall and NDCG performance. The models ranked consistently as follows:

$$\text{BERT} > \text{TF-IDF} > \text{BERTopic}$$

This confirms the effectiveness of deep semantic embeddings in capturing topical similarity for personalized recommendations.

For the ranking stage, we implemented a Learning-to-Rank (LETOR) model using XGBoost with LambdaMART, optimizing for the NDCG metric. This re-ranking strategy yielded measurable improvements in ranking quality, enhancing both early-stage recall and overall relevance. The NDCG@10 score of the BERT+DBSCAN model improved by 111% over the NCF-only baseline, and the Recall@10 increased by 145%.

Feature importance analysis revealed that social connections with project maintainers, developer programming proficiency, and profile similarity were the most influential predictors, outperforming static metadata. Interaction logits from the NCF model further strengthened the ranking model, with high-engagement signals (commits, pull requests) contributing more than lightweight actions (stars, forks). BERT-based semantic embeddings also proved valuable, indicating that contextual understanding of project descriptions complements collaborative and structural features.

Finally, our best-performing configuration, BERT embeddings with DBSCAN clustering, achieved a Recall@10 score of 0.21193. This indicates that the model is capable of recommending relevant projects early in the ranked list, successfully fulfilling our goal of helping developers discover meaningful OSS opportunities. This hybrid framework, integrating deep contextual embeddings, collaborative filtering, and learning-to-rank, represents a novel approach for OSS recommendation and holds promise for enhancing developer engagement in large-scale code ecosystems.

7.2 Future Work

In this study, we utilized a subset of the GitHub Torrent dataset (2010–2015) due to computational limitations, as processing the full dataset exceeds the capacity of a single machine. In real-world scenarios, the scale of data is significantly larger, and incorporating more comprehensive data could further improve recommendation accuracy. To address this, future work will explore the use of cloud-based middleware and distributed computing frameworks to enable efficient large-scale processing and training.

Additionally, we aim to enhance our feature set by incorporating real-world user interaction data through A/B testing. Capturing actual user behavior would help refine model performance and ensure more realistic personalization.

Furthermore, we will conduct sensitivity analyses on the NCF model by selectively dropping specific input features to evaluate their impact on performance. While feature importance has already been assessed through XGBoost, this additional step will offer deeper insight into the robustness and contribution of each feature within a neural collaborative filtering framework.

Ultimately, we intend to deploy the proposed model in a real-world recommender system, validating its effectiveness and adaptability in live environments.

Appendix A

GHTorrent Google BigQuery url

This appendix provides access to the GHTorrent dataset used in the experiments. GHTorrent Google BigQuery "ghtorrentmysql1906" can be found on: <https://console.cloud.google.com/bigqueryproject=ghtorrentmysql1906&p=ghtorrentmysql1906&d=MySQL1906&page=dataset&inv=1&inv=Ab3tYg&ws=!1m4!1m3!3m2!1sghtorrentmysql1906!2sMySQL1906>

Appendix B

Cluster Distribution

When we used TF-IDF tokenization and K-Means clustering, the analysis revealed that Cluster 28 contains the largest number of GitHub projects. By combining this observation with the corresponding keyword table, we can extract more targeted semantic features, ultimately boosting the precision and accuracy of the recommendation system model.

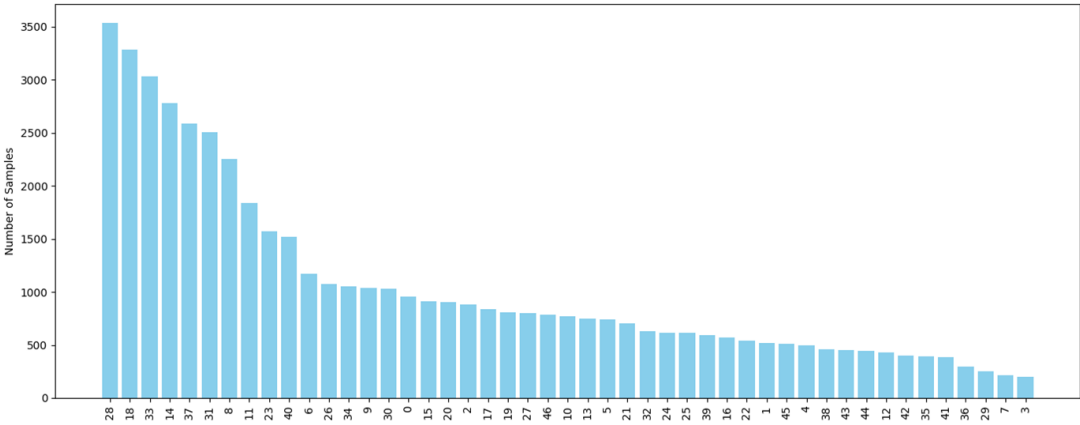


Figure B.1: GitHub Project Distribution Across 47 Clusters Generated via TF-IDF and K-Means Method

Cluster	Top Keywords	Cluster	Top Keywords
0	plugin, event driven, driven, android, based, tracker, event tracker, using, library, custom	24	official, git, git repository, main, code, project, mirror, main repository, official repository, code repository
1	web, repository, framework, web development, environment, development environment, development repository, software, project, application	25	javascript, written, java, language, compiler written, python, php, javascript compiler, using, library
2	management, platform, business logic, logic, intelligence, business intelligence, small business, small, process, web	26	chef, chef cookbook, repository, development, development repository, repository chef, opsource, opsource cookbook, repository opsource, cookbook install
3	official, based, simple, using, view, track, create, displays, ios, version	27	final, final project, django, data, management, tool, based, django project, using, repo
4	interface, line interface, tool, line tool, client, tools, line tools, utility, line client, line utility	28	library, simple, tool, based, using, module, management, rails, use, tools
5	pull, pull requests, http, http requests, library, github, repository, using, mirror, based	29	incubating, apache cordova, cordova, flink, apache flink, plugin, cordova plugin, apache hadoop, hadoop, apache maven
6	jquery, wordpress, jquery plugin, wordpress plugin, plugin wordpress, simple, text, plugin jquery, grunt, using	30	php, javascript, php framework, based, application, data, building, python, framework building, javascript framework
7	website, repository, project, code repository, android, framework, tool, site, library, documentation	31	api, php, library, client, wrapper, simple, rest, rest api, php library, data
8	library, python library, api, client, wrapper, implementation, python wrapper, python client, interface, simple	32	performance, distributed, high, high performance, data, based, database, framework, library, applications
9	app, web app, interface, applications, web applications, web interface, api, client, based, services	33	rails, android, rails app, django, django app, app android, mobile, google, simple, engine
10	library, application, android library, client, android application, android client, simple, library android, view, client android	34	api, rails, ruby rails, library, wrapper, ruby wrapper, gem, client, ruby gem, ruby client
11	client, library, api, module, javascript, nodejs client, browser, simple, library nodejs, client nodejs	35	javascript, python, library, ruby, generator, written, based, php, json, fast
12	engine, game engine, based, javascript, multiplayer, html, using, card, simple, project	36	html, javascript, html css, js, plugin, framework, using, css javascript, sass, files
13	like, library, data, api, project, plugin, designed, library provide, way, aims	37	library, client, api, java library, implementation, framework, java client, library java, written, simple
14	javascript, data, library, javascript library, implementation, client, tool, simple, analysis, using	38	library, adds, plugin, django, python, android, web, module, provides, extension
15	server, mirror, code, repository, based, thesis, template, project template	39	component, component zend, module, framework module, doctrine, module doctrine, repository, git repository, git, framework git
16	dropin, python, dropin replacement, android, support, module, provides, standard, based, classes	40	web, web application, django, rails, data, framework, management, simple, using, application framework
17	android, api, php, android sdk, javascript, nodejs, php sdk, azure, javascript sdk, python	41	chrome, chrome extension, yii, php, google, typo, google chrome, spree, web, typo extension
18	management, free, platform, data, open, engine, software, server, web, framework, api	42	framework, mvc framework, aspnet, aspnet mvc, javascript, php, nodejs, web, based, simple
19	api, client, web, nodejs, application, framework, simple, http, web server, client server	43	programming language, ruby, ruby programming, web, new, closure, closure programming, compiler, python, javascript
20	ouse, easy use, library, way, easy way, makes, make, android, makes easy, nodejs	44	wordpress, wordpress theme, bootstrap, theme wordpress, based, website, framework, starter, responsive, starter theme
21	python, project, tool, javascript, ruby, python code, used, using, api, data	45	framework perfectionists, perfectionists deadlines, deadlines, perfectionists, nodejs, php, framework nodejs, django, simple, django web
22	personal, personal website, new, code, source, project, repository, static, blog, official	46	python, ruby, python bindings, ruby bindings, nodejs, api, library, bindings nodejs, node, javascript
23	open, web, management, cross, cross platform, data, management platform, based, cloud, application		

Table B.1: Project Clustering Key Word Using TF-IDF and K-Means: 47 Group Overview

When we used BERT tokenization and DBSCAN clustering, the analysis revealed that Cluster 7 contains the highest number of GitHub projects, indicating its dominant role within the data distribution. Although each cluster is relatively balanced in size, Cluster -1 includes only a small number of instances, 1,386, which is beneficial for maintaining cluster quality. By combining these structural insights with the keyword table, we can extract more targeted semantic features and, in future work, improve the precision and accuracy of the recommendation system model.

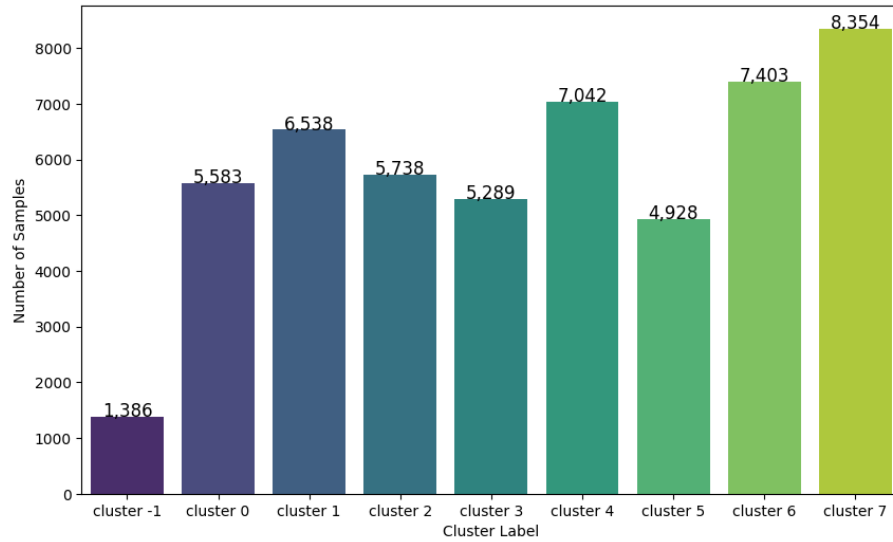


Figure B.2: GitHub Project Distribution Across 8 Clusters Generated via BERT and DBSCAN Method

Cluster	Top Keywords
0	code, source, website, blog, jsonlintcom, iteamse, toxim, tutorials, lionscom, preview gostyle, concurrency, library, lisp, uboot, gpu, flash, pebble, gcc, lg
1	pipeline, asset, rails, use, library, templates, javascript, jquery, integrate, editor text, sublime, plugin, settings, xiki, jsfmt, livestyle, latex, coldfusion, detector
2	ui, project, phone, htmlbased, gecko, gaia, boot, used, releases, note api, ruby, wrapper, linkedin, foursquare, rest, bitly, simple, web, google
3	component, framework, zend, module, doctrine, escaper, stdlib, log, db, feed chrome, extension, google, seconds, extensionizr, create, scaffold, fiddler, anyfetch, shanbaycom
4	programming, language, clojure, boo, kotlin, emojiased, completely, javascript, elsa, duck code, source, website, blog, jsonlintcom, iteamse, toxim, tutorials, lionscom, preview
5	interface, python, api, model, bitcoins, jsonrpc, transfer, radiative, library, code apache, mirror, usergrid, james, ant, aurora, calcite, directmemory, felix, flink
6	wrapper, python, api, library, gitlab, simple, rest, nhs, serving, ods rails, ruby, weeks, paris, entrepreneurs, bootcamp, template, plugin, website, clientsidevalidations
7	api, ruby, wrapper, linkedin, foursquare, rest, bitly, simple, web, google, coinomi android, numberpicker, companion, backport, preference, anyfetch, homebrewers, watcher, decompiler,

Table B.2: Project Clustering Key Word Using BERT and DBSCAN: 8 Group Overview

Appendix C

BERT Distribution

This figure demonstrates the distribution of GitHub projects clustered using BERT token embeddings combined with DBSCAN. The visualization uses distinct colors to represent different clusters.

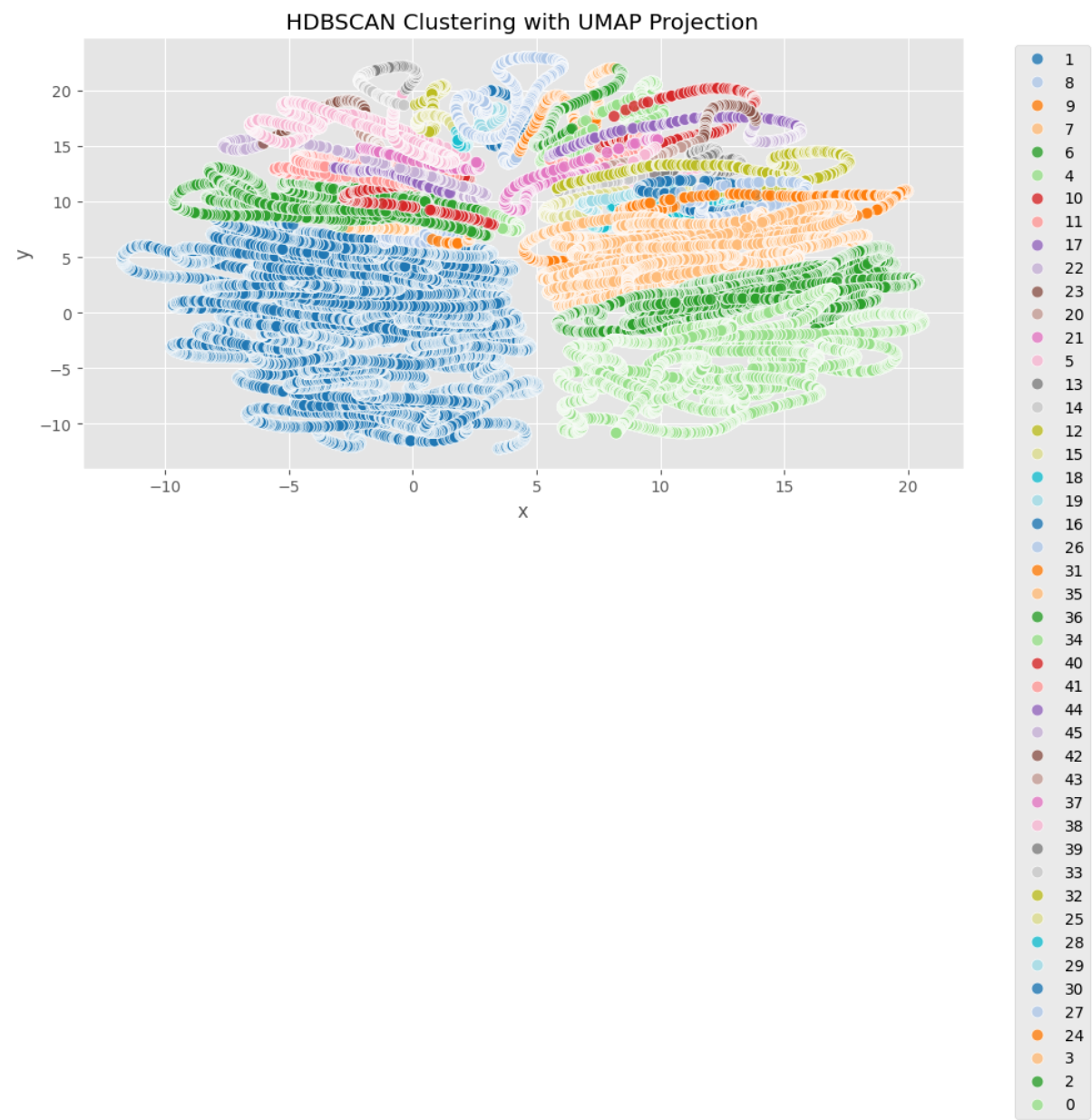


Figure C.1: HDBSCAN clustering results using UMAP projection of BERT-based tokenized project descriptions.

Bibliography

- Anelli, V. W., Bellogín, A., Di Noia, T., Jannach, D., and Pomo, C. (2022). Top-n recommendation algorithms: A quest for the state-of-the-art. In *Proceedings of the 30th ACM Conference on User Modeling, Adaptation and Personalization (UMAP '22)*, pages 1–11, Barcelona, Spain. ACM. Published in ACM, New York, NY, USA.
- Bai, S., Liu, L., Liu, H., Zhang, M., Meng, C., and Zhang, P. (2022). Find potential partners: A github user recommendation method based on event data. *Information and Software Technology*, 150:106961.
- Bengio, Y., Lamblin, P., Popovici, D., and Larochelle, H. (2007). Greedy layer-wise training of deep networks. In Schölkopf, B., Platt, J., and Hofmann, T., editors, *Advances in Neural Information Processing Systems 19 (NeurIPS 2006)*. MIT Press. Published in Proceedings of NeurIPS 2006.
- Bentéjac, C., Csörgő, A., and Martínez-Muñoz, G. (2021). A comparative analysis of gradient boosting algorithms. *Artificial Intelligence Review*, 54(3):1937–1967.
- Blincoe, K., Sheoran, J., Goggins, S., Petakovic, E., and Damian, D. (2016). Understanding the popular users: Following, affiliation influence and leadership on github. *Information and Software Technology*, 70:30–39.
- Burges, C. J. (2010). From ranknet to lambdarank to lambdamart: An overview. Technical Report MSR-TR-2010-82.
- Chacon, S. and Straub, B. (2014). *Github*, pages 131–180. Apress, Berkeley, CA.
- Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system. *CoRR*, abs/1603.02754.
- Chen, Z., Guo, J., and Liu, Q. (2018). Dbscan algorithm clustering for massive ais data based on the hadoop platform. In *Proceedings of the International Conference on Industrial Informatics – Computing Technology, Intelligent Technology, Industrial Information Integration (ICIICII)*, pages 25–28.
- Covington, P., Adams, J., and Sargin, E. (2016). Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems, RecSys '16*, page 191–198, New York, NY, USA. Association for Computing Machinery.
- Dabbish, L., Stuart, C., Tsay, J., and Herbsleb, J. (2012). Social coding in github: transparency and collaboration in an open software repository. In *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work, CSCW '12*, page 1277–1286, New York, NY, USA. Association for Computing Machinery.

- Davies, D. L. and Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227.
- Devlin, J., Chang, M., Lee, K., and Toutanova, K. (2018). BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding.
- Di Rocco, J., Di Ruscio, D., Di Sipio, C., Nguyen, P., and Rubei, R. (2023). Hybridrec: A recommender system for tagging github repositories. *Applied Intelligence*, 53(8):9708–9730.
- Eluri, V. K., Mazzuchi, T. A., and Sarkani, S. (2021). Predicting long-time contributors for github projects using machine learning. *Information and Software Technology*, 138:106616. Accessed July 2025.
- Enriquez-Loja, J., Castillo-Pérez, B., Serrano-Guerrero, X., and Barragán-Escandón, A. (2025). Performance evaluation method for different clustering techniques. *Computers and Electrical Engineering*, 123:110132.
- Ezugwu, A. E., Ikotun, A. M., Oyelade, O. O., Abualigah, L., Agushaka, J. O., Eke, C. I., and Akinyelu, A. A. (2022). A comprehensive survey of clustering algorithms: State-of-the-art machine learning applications, taxonomy, challenges, and future research prospects. *Engineering Applications of Artificial Intelligence*, 110:104743.
- Fareed, A., Hassan, S., Belhaouari, S. B., and Halim, Z. (2023). A collaborative filtering recommendation framework utilizing social networks. *Machine Learning with Applications*, 14:100495.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.
- Gao, C., Zheng, Y., Li, N., Li, Y., Qin, Y., Piao, J., Quan, Y., Chang, J., Jin, D., He, X., and Li, Y. (2023). A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Trans. Recomm. Syst.*, 1(1).
- Garrido-Merchan, E. C., Gozalo-Brizuela, R., and Gonzalez-Carvajal, S. (2023). Comparing bert against traditional machine learning models in text classification. *Journal of Computational and Cognitive Engineering*, 2(4):352–356.
- GitHub Documentation Team (2022). Github event types. <https://docs.github.com/en/rest/using-the-rest-api/github-event-types?apiVersion=2022-11-28>. Accessed: July 25, 2025.
- Gousios, G. and Spinellis, D. (2012). Ghtorrent: Github’s data from a firehose. In *2012 9th IEEE Working Conference on Mining Software Repositories (MSR)*, pages 12–21.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data?

- Grootendorst, M. (2022). Bertopic: Neural topic modeling with a class-based tf-idf procedure.
- Guendouz, M., Amine, A., and Hamou, R. M. (2015). Recommending relevant open source projects on github using a collaborative-filtering technique. *International Journal of Open Source Software and Processes*, 6(1):1–16.
- Hahn, J., Moon, J. Y., and Zhang, C. (2008). Emergence of new project teams from open source software developer networks: Impact of prior collaboration ties. *Information Systems Research*, 19(3):369–391.
- Han, J., Deng, S., Xia, X., Wang, D., and Yin, J. (2019). Characterization and prediction of popular projects on github. In *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, volume 1, pages 21–26.
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. (2017). Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web, WWW '17*, page 173 – 182, Republic and Canton of Geneva, CHE. International World Wide Web Conferences Steering Committee.
- Hegde, T., Sanjay, K. S., Thomas, S. M., Kambhammettu, R., Anand Kumar, M., and Ramanna, S. (2023). Impact of vector embeddings on the performance of tolerance near sets-based sentiment classifier for text classification. *Procedia Computer Science*, 225:645–654. 27th International Conference on Knowledge Based and Intelligent Information and Engineering Systems (KES 2023).
- Hevner, A. R., March, S. T., Park, J., and Ram, S. (2004). Design science in information systems research. *MIS Q.*, 28(1):75–105.
- Hu, Y., Wang, S., Ren, Y., and Choo, K.-K. R. (2018). User influence analysis for github developer social networks. *Expert Systems with Applications*, 108:108–118.
- Jamshidi, S., Mohammadi, M., Bagheri, S., Najafabadi, H. E., Rezvanian, A., Gheisari, M., Ghaderzadeh, M., Shahabi, A. S., and Wu, Z. (2024). Effective text classification using bert, mtm lstm, and dt. *Data and Knowledge Engineering*, 151:102306.
- Jawansingh, R. S. and Rai, A. K. (2025). Evaluating recommender system using character approaches. *Procedia Computer Science*, 258:2323–2333. International Conference on Machine Learning and Data Engineering.
- Jiang, J., Lo, D., Yang, Y., Li, J., and Zhang, L. (2019). A first look at unfollowing behavior on github. *Information and Software Technology*, 105:150–160.
- Jiang, L.-C., Liu, R.-R., and Jia, C.-X. (2022). User-location distribution serves as a useful feature in item-based collaborative filtering. *Physica A: Statistical Mechanics and its Applications*, 586:126491.
- Jin, W., Yan, L., Jiang, J., and Yu, Y. (2023). An object association matching method based on v2i system. *International Journal of Computational Intelligence Systems*, 16:132.

- Kale, A., Khanvilkar, O., Jivani, H., Kumkar, P., Madan, I., and Sarode, T. (2018). Forecasting indian stock market using artificial neural networks. In *2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA)*, pages 1–5.
- Kingma, D. P. and Ba, J. L. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. Published as a conference paper at ICLR 2015.
- Kobayakawa, N. and Yoshida, K. (2019). Study on influencers of cryptocurrency follow-network on github. In Ohara, K. and Bai, Q., editors, *Knowledge Management and Acquisition for Intelligent Systems*, pages 173–183, Cham. Springer International Publishing.
- Koskela, M., Simola, I., and Stefanidis, K. (2018). Open source software recommendations using github. In Méndez, E., Crestani, F., Ribeiro, C., David, G., and Lopes, J. C., editors, *Digital Libraries for Open Knowledge (TPDL)*, volume 11057 of *Lecture Notes in Computer Science*, pages 279–285. Springer, Cham.
- Li, P. L., Ko, A. J., and Begel, A. (2017). Cross-disciplinary perspectives on collaborations with software engineers. In *2017 IEEE/ACM 10th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*, pages 2–8.
- Li, X., Moreschini, S., Zhang, Z., and Taibi, D. (2022). Exploring factors and metrics to select open source software components for integration: An empirical study. *Journal of Systems and Software*, 188:111255.
- Lima, A., Rossi, L., and Musolesi, M. (2014). Coding together at scale: Github as a collaborative social network. In *Proceedings of the 8th International Conference on Weblogs and Social Media (ICWSM)*, Ann Arbor, Michigan, USA. AAAI.
- Liu, C., Yang, D., Zhang, X., Ray, B., and Rahman, M. M. (2018a). Recommending github projects for developer onboarding. *IEEE Access*, 6:52082–52094.
- Liu, C., Yang, D., Zhang, X., Ray, B., and Rahman, M. M. (2018b). Recommending github projects for developer onboarding. *IEEE Access*, 6:52082–52094.
- Lucchese, C., Nardini, F. M., Orlando, S., Perego, R., and Veneri, A. (2025). Explainable, effective, and efficient learning-to-rank models using ilmart. *ACM Transactions on Information Systems*, 43(3):105.
- Malzer, C. and Baum, M. (2020). A hybrid approach to hierarchical density-based cluster selection. In *2020 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*, page 223–228. IEEE.
- Matek, T. and Zebec, S. T. (2016). Github open source project recommendation system.
- Mohamed, A. (2020). An effective dimension reduction algorithm for clustering arabic text. *Egyptian Informatics Journal*, 21(1):1–5.

- Nalini, N., Rishabh, S., Bhat, B. D., Jamwal, V., and Kumar, A. (2023). Github recommendation system and user analytics. In *2023 9th International Conference on Smart Computing and Communications (ICSCC)*, pages 582–587.
- Nourry, O., Kondo, M., Saito, S., Iimura, Y., Ubayashi, N., and Kamei, Y. (2024). Myth: The loss of core developers is a critical issue for oss communities. arXiv preprint, Version 1. License: arXiv.org perpetual non-exclusive.
- Nugroho, Y. S., Islam, S., Gunawan, D., Kurniawan, Y. I., and Hossain, M. J. (2022). Dataset of network simulator related-question posts in stack overflow. *Data in Brief*, 41:107942.
- Onoue, S., Hata, H., and Matsumoto, K.-i. (2013). A study of the characteristics of developers’ activities in github. In *2013 20th Asia-Pacific Software Engineering Conference (APSEC)*, volume 2, pages 7–12.
- Porter, M. (1980). An algorithm for suffix stripping. *Program: Electronic Library and Information Systems*, 14(3):130–137.
- Qomariyah, N. N., Kazakov, D., and Fajar, A. N. (2020). Predicting user preferences with xgboost learning to rank method. In *2020 3rd International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*, pages 123–128.
- Raza, S., Rahman, M., Kamawal, S., Toroghi, A., Raval, A., Navah, F., and Kazemeini, A. (2025). A comprehensive review of recommender systems: Transitioning from theory to practice.
- Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., and Riedl, J. (1994). Grouplens: an open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work, CSCW '94*, page 175–186, New York, NY, USA. Association for Computing Machinery.
- Ricci, F., Rokach, L., and Shapira, B. (2011). Introduction to recommender systems handbook. In *Recommender Systems Handbook*, pages 1–35. Springer US, Boston, MA.
- Sarwar, B., Karypis, G., Konstan, J., and Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web, WWW '01*, page 285–295, New York, NY, USA. Association for Computing Machinery.
- Seth, R. and Sharaff, A. (2022). *A Comparative Overview of Hybrid Recommender Systems: Review, Challenges, and Prospects*, pages 57–98.
- Steinmacher, I., Graciotto Silva, M. A., Gerosa, M. A., and Redmiles, D. F. (2015). A systematic literature review on the barriers faced by newcomers to open source software projects. *Information and Software Technology*, 59:67–85.
- Sun, X., Yang, H., Xia, X., and Li, B. (2017). Enhancing developer recommendation with supplementary information via mining historical commits. *Journal of Systems and Software*, 134:355–368.

- Taha, K., Yoo, P. D., Yeun, C., Homouz, D., and Taha, A. (2024). A comprehensive survey of text classification techniques and their research applications: Observational and experimental insights. *Computer Science Review*, 54:100664.
- Tiburcio, P. G. and Spohn, M. A. (2010). Ad hoc grid: An adaptive and self-organizing peer-to-peer computing grid. In *2010 10th IEEE International Conference on Computer and Information Technology*, pages 225–232.
- Vasilescu, B., Filkov, V., and Serebrenik, A. (2015). Perceptions of diversity on git hub: A user survey. In *2015 IEEE/ACM 8th International Workshop on Cooperative and Human Aspects of Software Engineering*, pages 50–56.
- Wen, X., Cui, P., Luo, Y., Hu, R., and Guo, Y. (2025). A learning-to-rank method to identify crash hotspots based on large-scale ride-hailing crash data. *Multimodal Transportation*, 4(3):100219.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. (2020). Transformers: State-of-the-art natural language processing. In Liu, Q. and Schlangen, D., editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.
- Wu, H. C., Luk, R. W. P., Wong, K. F., and Kwok, K. L. (2008). Interpreting tf-idf term weights as making relevance decisions. *ACM Trans. Inf. Syst.*, 26(3).
- Yang, H., Sun, S., Wen, J., Cai, H., and Mateen, M. (2021). Improving personalized project recommendation on github based on deep matrix factorization. In *Proceedings of the International Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom)*, volume 406 of *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, pages 318–332. Springer.
- Yue, Z., Rabhi, S., de Souza Pereira Moreira, G., Wang, D., and Oldridge, E. (2018). Llamarec: Two-stage recommendation using large language models for ranking. In *Proceedings of Woodstock '18: ACM Symposium on Neural Gaze Detection*, pages 1–6, Woodstock, NY, USA. ACM.
- Zahra, S., Ghazanfar, M. A., Khalid, A., Azam, M. A., Naeem, U., and Prugel-Bennett, A. (2015). Novel centroid selection approaches for kmeans-clustering based recommender systems. *Information Sciences*, 320:156–189.
- Zhang, L., Zou, Y., Xie, B., and Zhu, Z. (2014). Recommending relevant projects via user behaviour: an exploratory study on github. In *Proceedings of the 1st International Workshop on Crowd-Based Software Development Methods and Technologies*, CrowdSoft 2014, page 25–30, New York, NY, USA. Association for Computing Machinery.
- Zhang, Y., Xu, F. F., Li, S., Meng, Y., Wang, X., Li, Q., and Han, J. (2019). Higitclass: Keyword-driven hierarchical classification of github repositories. In *2019 IEEE International Conference on Data Mining (ICDM)*, pages 876–885.

- Zhang, Z., Ni, G., and Xu, Y. (2020). Comparison of trajectory clustering methods based on k-means and dbSCAN. In *Proceedings of the IEEE International Conference on Big Data and Artificial Intelligence (ICIBA)*, pages 557–561.